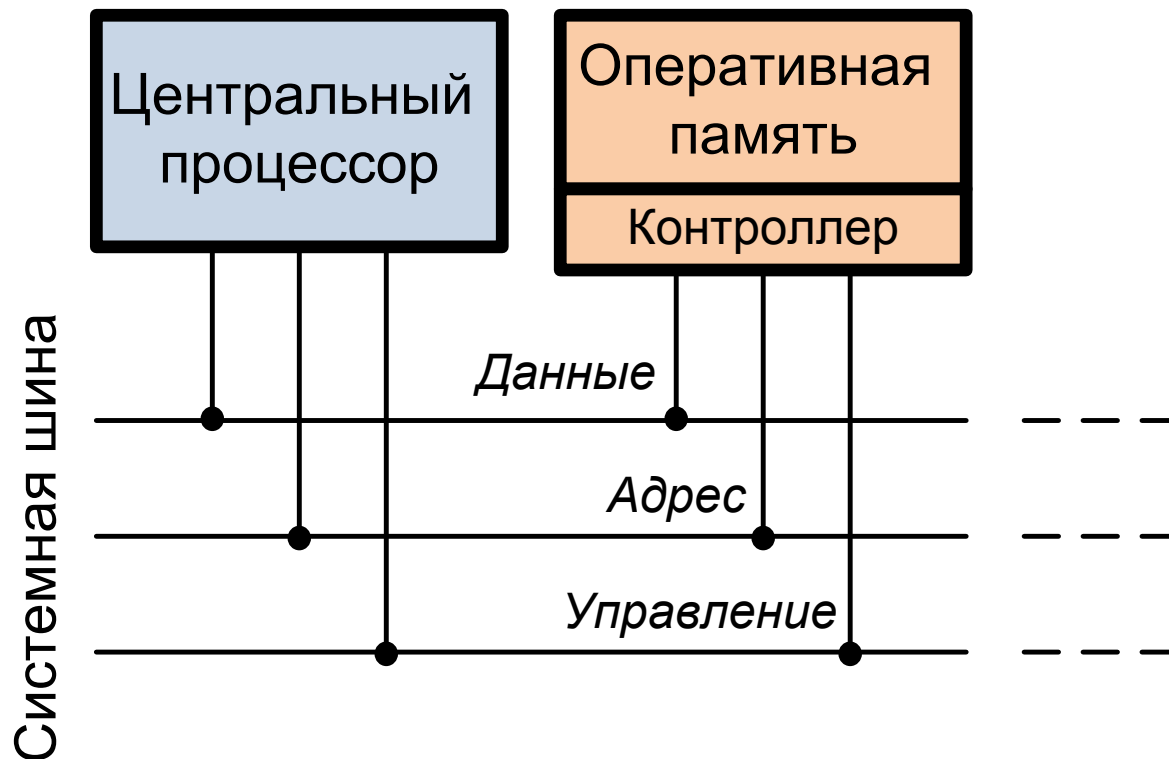


**Взаимодействие устройств
ЭВМ в процессе
выполнения машинной
команды
(рабочий цикл процессора)**

В каждый момент времени компьютер выполняет определенную программу. Программа – последовательность действий (команд), направленная на преобразование некоторых входных данных в выходные (результат).

Команды и данные программы, исполняемой в текущий момент времени хранятся в оперативной памяти (ОП). Выполняет команды процессор.



Вопрос: В каком виде хранятся данные и команды?

Откуда процессор знает, где в ОП находится очередная команда и данные для нее?

Представление данных

(подробно рассматривали в теме АЛУ)

$$\begin{array}{cccccccc}
 3\text{н} & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\
 \boxed{0} & \boxed{1} & \boxed{1} & \boxed{0} & \boxed{1} & \boxed{1} & \boxed{0} & \boxed{1}
 \end{array}
 = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 1 \cdot 2^3 + 0 \cdot 2^4 + 1 \cdot 2^5 + 1 \cdot 2^6 =$$

$$= 1 + 4 + 8 + 32 + 64 = +109$$

$\left[\begin{array}{l} 0 - \text{число положительное,} \\ 1 - \text{число отрицательное} \end{array} \right.$

Числа представляются в ЭВМ в двоичной системе счисления (форматы со знаком с фиксированной и с плавающей точкой).

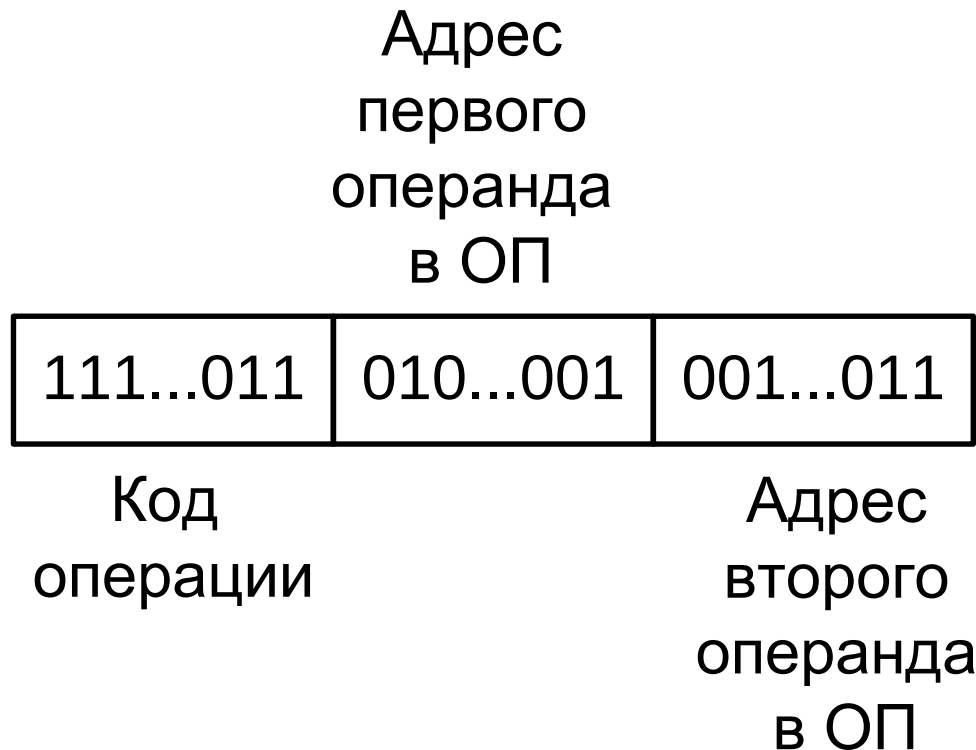
Символы представляются их двоичными кодами.

Двоичные вектора (над которыми производятся поразрядные логические операции) представляются последовательностями битов, для хранения которых, как правило, используются беззнаковые целые типы.

Представление команд

Команда – это приказ компьютеру на выполнение какой-либо операции, например, операции сложения двух чисел (операндов), которые хранятся в оперативной памяти.

Можно представить команду в следующем формате:



Результат, как правило, помещается в память на место первого операнда

Каждый процессор исполняет определенный набор команд, который называется системой команд процессора. В этот набор входят арифметические и логические команды, команды пересылки информации между регистрами процессора и оперативной памятью, команды передачи управления, команды ввода-вывода и др.

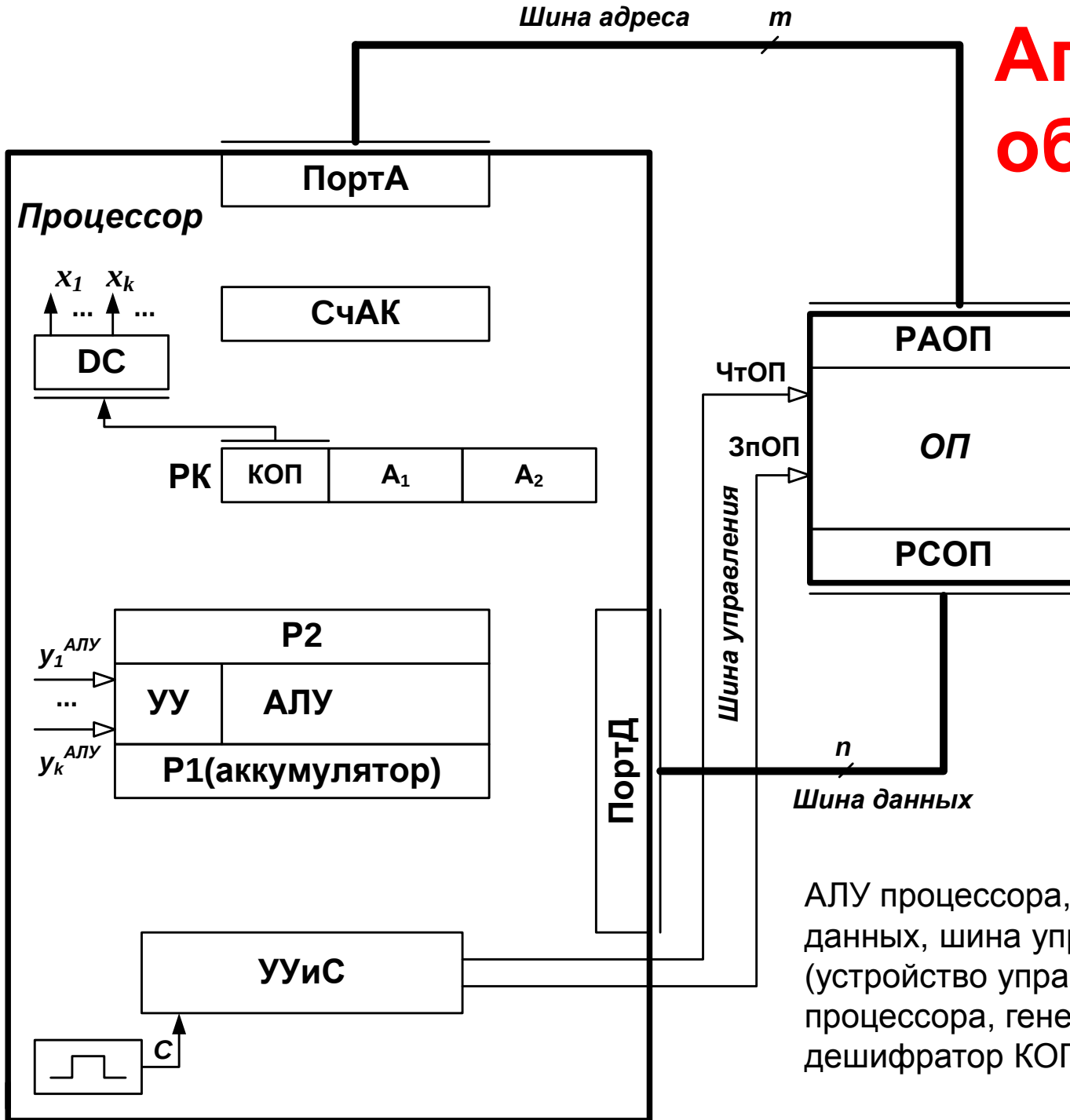
Данные о системе команд конкретного процессора можно получить из соответствующих справочников.

В качестве примера приведем фрагмент описания системы команд микропроцессора (микроконтроллера) КМ1816ВЕ51.

Таблица 3.13. Группа команд арифметических операций

Название команды	Мнемокод	КОП	Т	Б	Ц	Операция
Сложение аккумулятора с регистром ($n = 0 \div 7$)	ADD A, Rn	00101rrr	1	1	1	$(A) \leftarrow (A) + (Rn)$
Сложение аккумулятора с прямоадресуемым байтом	ADD A, ad	00100101	3	2	1	$(A) \leftarrow (A) + (ad)$
Сложение аккумулятора с байтом из РПД ($i = 0..1$)	ADD A, @Ri	0010011i	1	1	1	$(A) \leftarrow (A) + ((Ri))$
Сложение аккумулятора с константой	ADD A, #d	00100100	2	2	1	$(A) \leftarrow (A) + \#d$
Сложение аккумулятора с регистром и переносом	ADDC A, Rn	00111rrr	1	1	1	$(A) \leftarrow (A) + (Rn) + (C)$
Сложение аккумулятора с прямоадресуемым байтом и переносом	ADDC A, ad	00110101	3	2	1	$(A) \leftarrow (A) + (ad) + (C)$
Сложение аккумулятора с байтом из РПД и переносом	ADDC A, @Ri	0011011i	1	1	1	$(A) \leftarrow (A) + ((Ri)) + (C)$
Сложение аккумулятора с константой и переносом	ADDC A, #d	00110100	2	2	1	$(A) \leftarrow (A) + \#d + (C)$
Десятичная коррекция аккумулятора	DA A	11010100	1	1	1	Если $(A_{0-3}) > 9 \vee ((AC) = 1)$, то $(A_{0-3}) \leftarrow (A_{0-3}) + 6$, затем если $(A_{4-7}) > 9 \vee ((C) = 1)$, то $(A_{4-7}) \leftarrow (A_{4-7}) + 6$
Вычитание из аккумулятора регистра и заема	SUBB A, Rn	10011rrr	1	1	1	$(A) \leftarrow (A) - (C) - (Rn)$
Вычитание из аккумулятора прямоадресуемого байта и заема	SUBB A, ad	10010101	3	2	1	$(A) \leftarrow (A) - (C) - ((ad))$
Вычитание из аккумулятора байта РПД и заема	SUBB A, @Ri	1001011i	1	1	1	$(A) \leftarrow (A) - (C) - ((Ri))$
Вычитание из аккумулятора константы и заема	SUBB A, #d	10010100	2	2	1	$(A) \leftarrow (A) - (C) - \#d$
Инкремент аккумулятора	INC A	00000100	1	1	1	$(A) \leftarrow (A) + 1$
Инкремент регистра	INC Rn	00001rrr	1	1	1	$(Rn) \leftarrow (Rn) + 1$
Инкремент прямоадресуемого байта	INC ad	00000101	3	2	1	$(ad) \leftarrow (ad) + 1$
Инкремент байта в РПД	INC @Ri	0000011i	1	1	1	$((Ri)) \leftarrow ((Ri)) + 1$
Инкремент указателя данных	INC DPTR	10100011	1	1	2	$(DPTR) \leftarrow (DPTR) + 1$
Декремент аккумулятора	DEC A	00010100	1	1	1	$(A) \leftarrow (A) - 1$
Декремент регистра	DEC Rn	00011rrr	1	1	1	$(Rn) \leftarrow (Rn) - 1$
Декремент прямоадресуемого байта	DEC ad	00010101	3	2	1	$(ad) \leftarrow (ad) - 1$
Декремент байта в РПД	DEC @Ri	0001011i	1	1	1	$((Ri)) \leftarrow ((Ri)) - 1$
Умножение аккумулятора на регистр B	MUL AB	10100100	1	1	4	$(B)(A) \leftarrow (A) \times (B)$
Деление аккумулятора на регистр B	DIV AB	10000100	1	1	4	$(A).(B) \leftarrow (A)/(B)$

Аппаратное обеспечение



Регистры процессора:
РК – регистр команд;
ПортА – порт адреса;
ПортД – порт данных;
Р1 – аккумулятор АЛУ;
Р2 – второй регистр АЛУ;
СчАК – счетчик адреса команды.

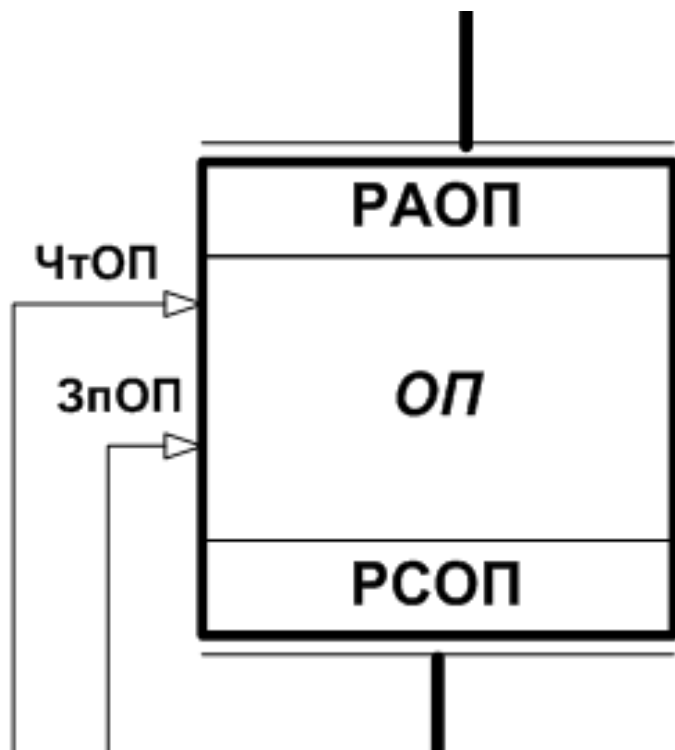
Регистры оперативной памяти (ОП):
РАОП – регистр адреса ОП;
РСОП – регистр слова ОП.

АЛУ процессора, ОП, шина адреса, шина данных, шина управления, УУиС (устройство управления и синхронизации) процессора, генератор синхросерии С, дешифратор КОП (DC).

Устройства, входящие в состав процессора

- 1) УУиС – устройство управления и синхронизации (управляет работой всех других устройств);
- 2) СчАК – счетчик адреса команды (длина определяется количеством битов адреса в системе, зависящим от объема оперативной памяти в байтах) хранит адрес очередной команды, которая должна быть считана из памяти и выполнена ;
- 3) РК – регистр команд (длина определяется максимальной длиной команды) хранит выбранную из памяти команду на всем протяжении ее выполнения процессором;
- 4) АЛУ, имеющее два регистра для операндов, один из которых является аккумулятором, т.е. регистром, в который загружается операнд, а потом в нем появляется результат выполнения операции в АЛУ (длина регистров определяется максимальной длиной операнда);
- 5) ПортА подключен к шине адреса, которая, в свою очередь подключена к регистру адреса оперативной памяти РАОП (длина равна количеству битов (линий) шины адреса).
- 6) ПортД подключен к шине данных, которая, в свою очередь, подключена к регистру слова оперативной памяти РСОП (длина равна количеству битов (линий) шины данных).

... (продолжим список позже).



Чтобы осуществить считывание данных из ОП процессор должен выставить на шину адреса (в РАОП) адрес слова ОП, в котором хранятся данные, и выработать управляющий сигнал ЧтОП. Считанные данные появляются в РСОП и поступают по шине данных в порт данных.

Чтобы осуществить запись данных в ОП, процессор должен выставить на шину адреса (в РАОП) адрес слова ОП, в которое нужно записать данные, на шину данных (в РСОП) – сами данные, и выработать управляющий сигнал (ЗпОП).

Рабочий цикл процессора

на примере выполнения двухадресной арифметической (или логической) команды.

Рассматриваем упрощенный вариант, но достаточный для понимания принципа взаимодействия процессора и ОП в процессе выполнения машинной команды.

Адрес
первого
операнда
в ОП

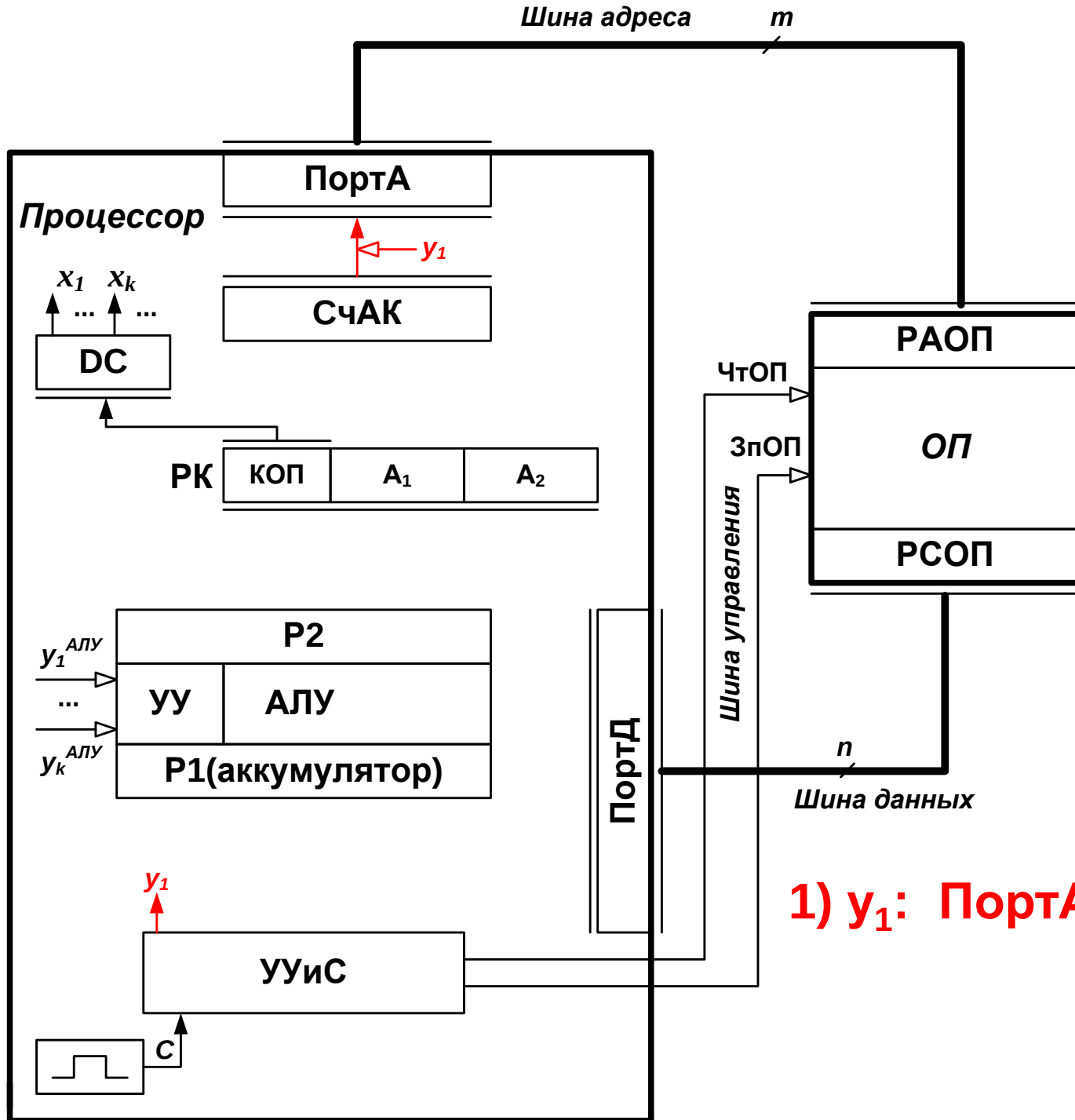
111...011	010...001	001...011
-----------	-----------	-----------

Код
операции

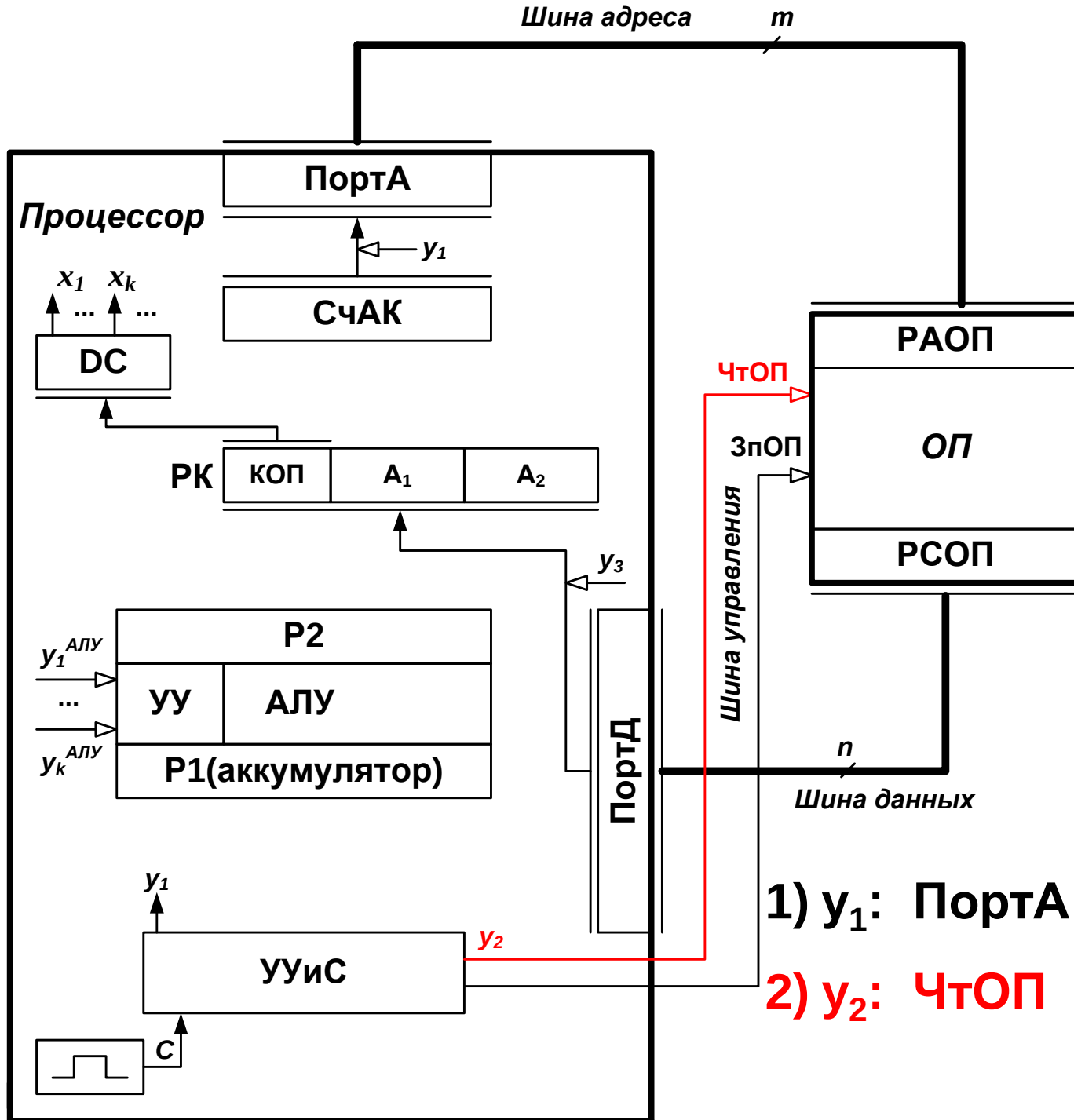
Адрес
второго
операнда
в ОП

1 этап.

Выборка команды из ОП

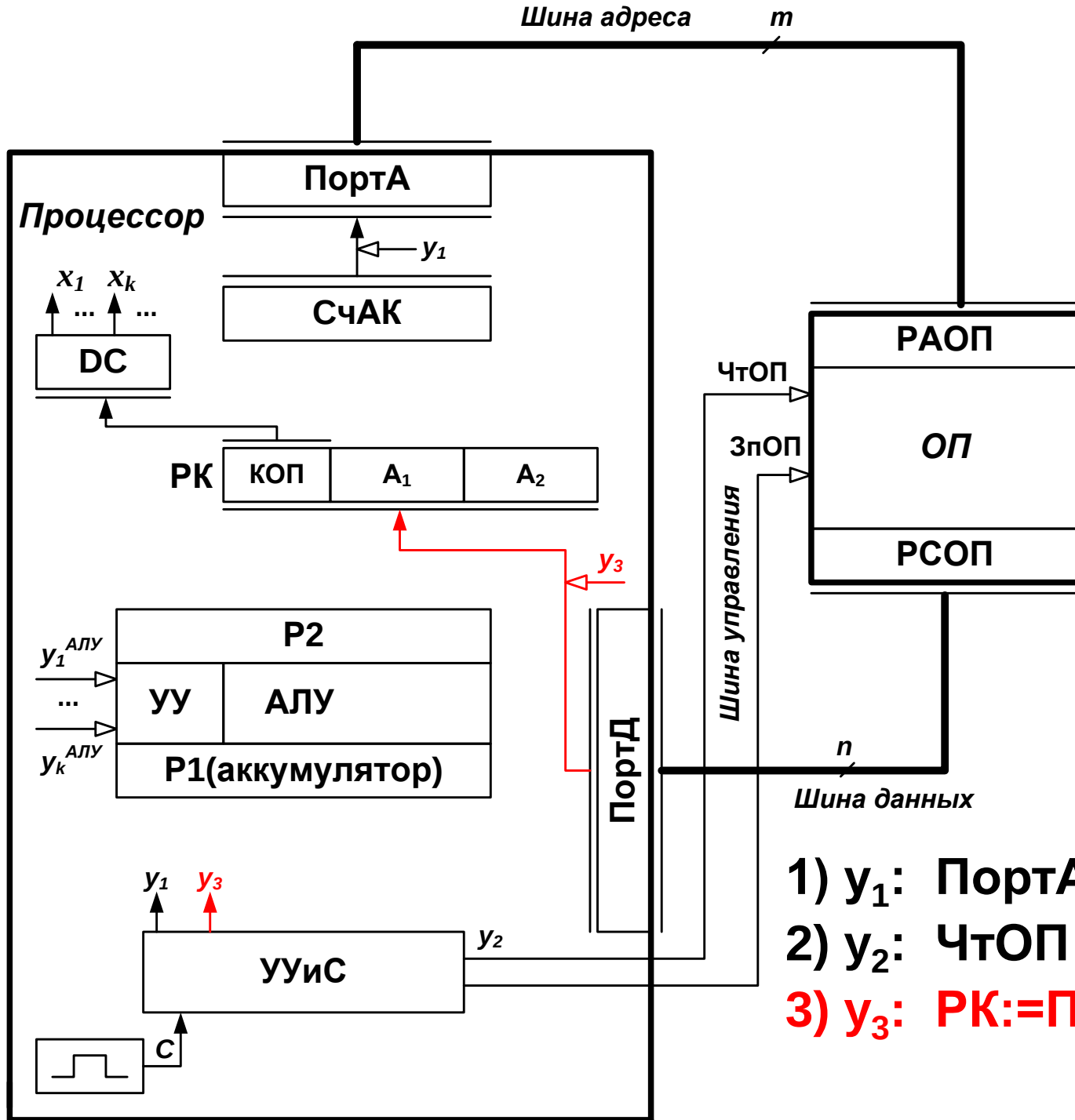


1) y_1 : ПортА:=СчАК



1) y_1 : ПортА:=СЧАК

2) y_2 : ЧТОП



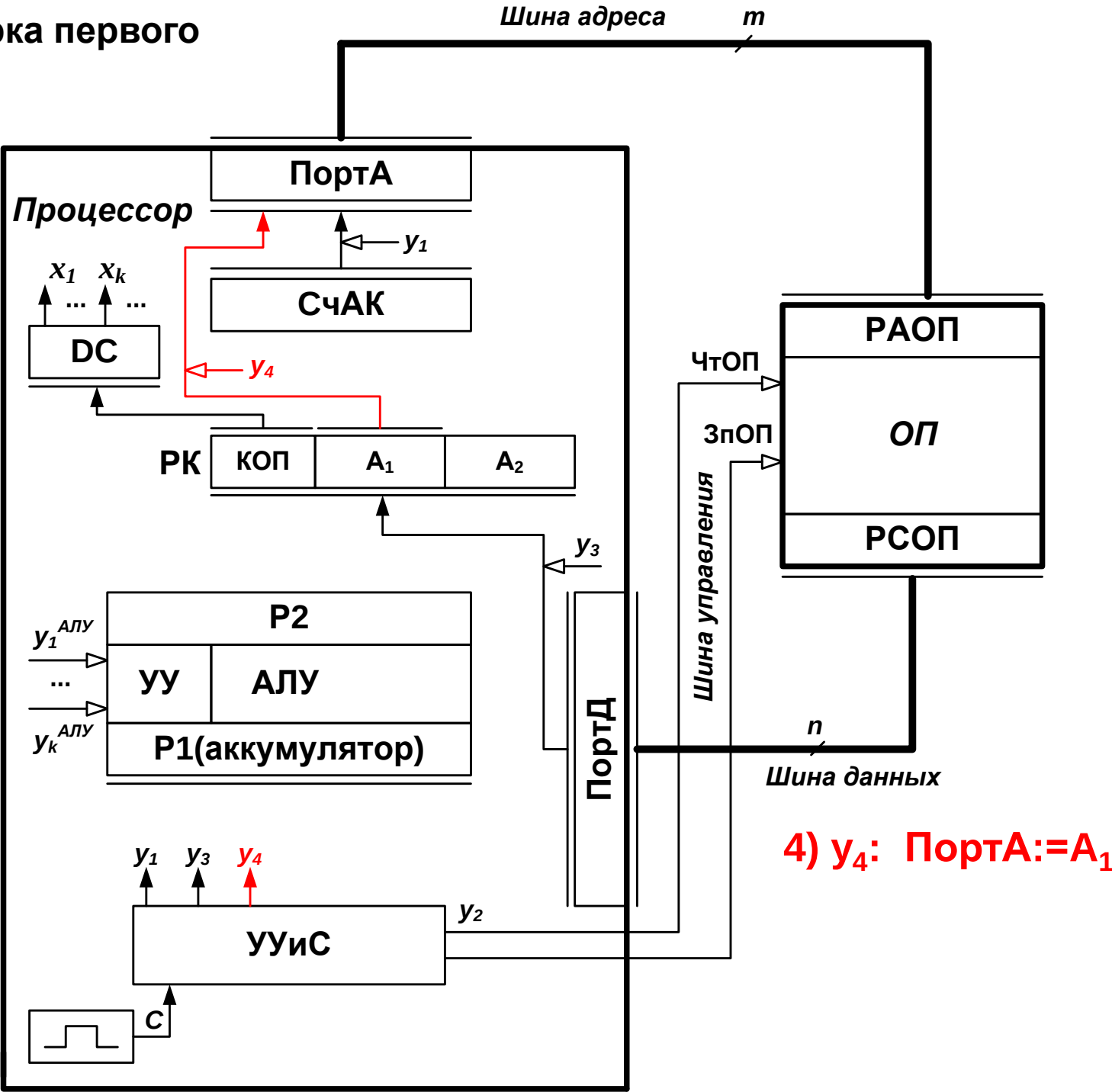
- 1) y_1 : ПортА:=СчАК
- 2) y_2 : ЧТОП
- 3) y_3 : РК:=ПортД

Рабочий цикл процессора

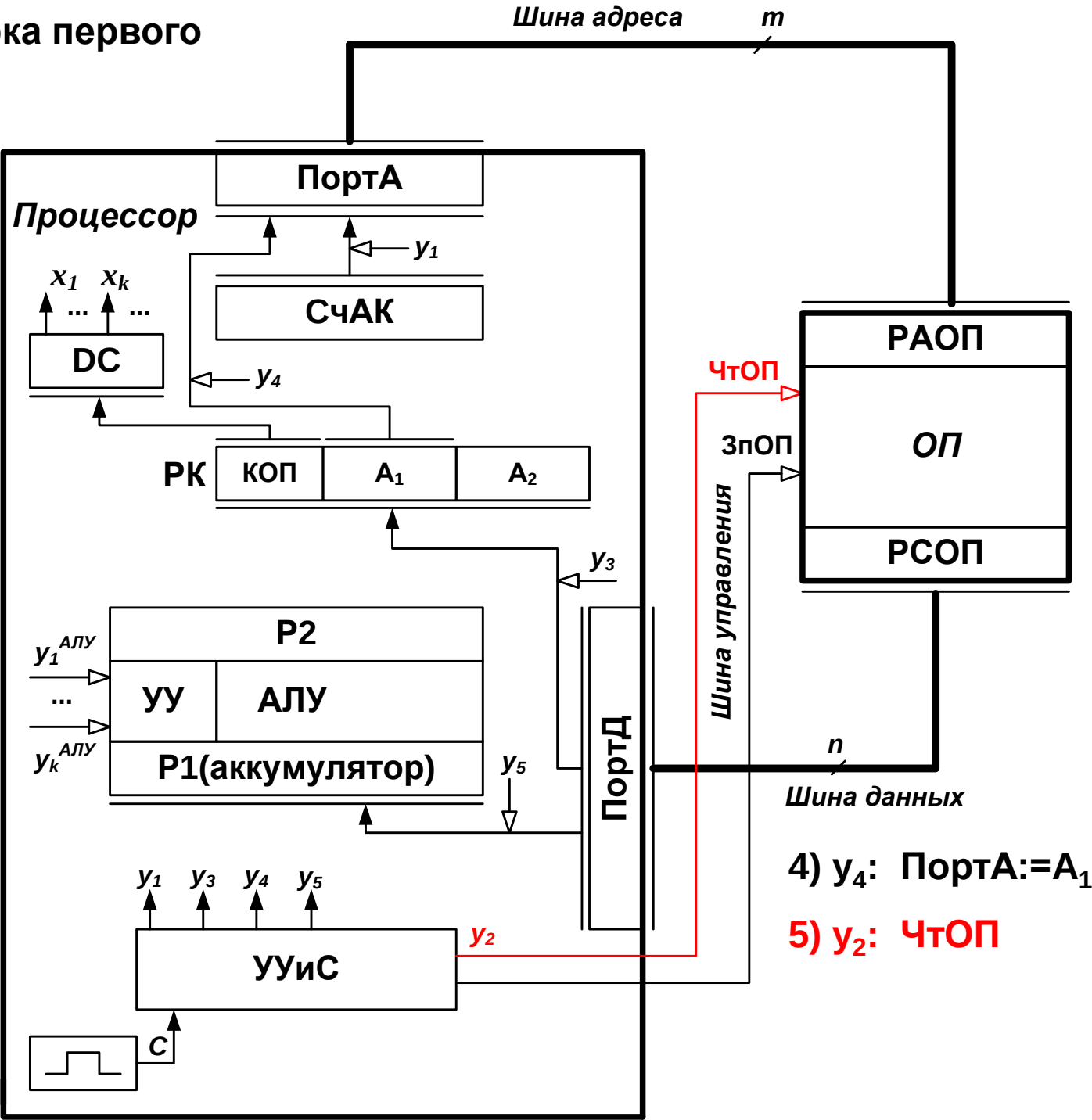
2 этап.

Выборка операндов из ОП

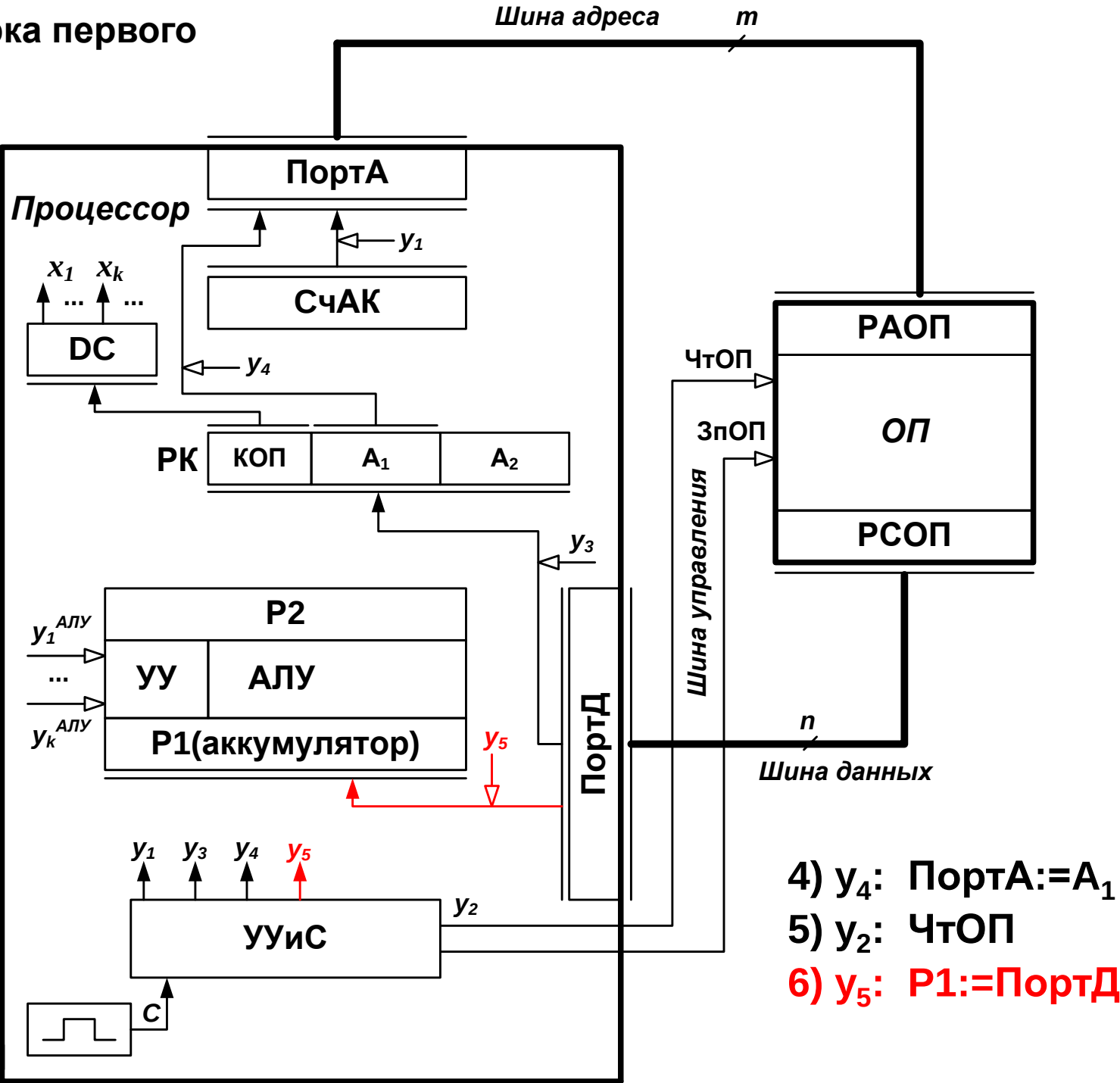
2.1. Выборка первого операнда



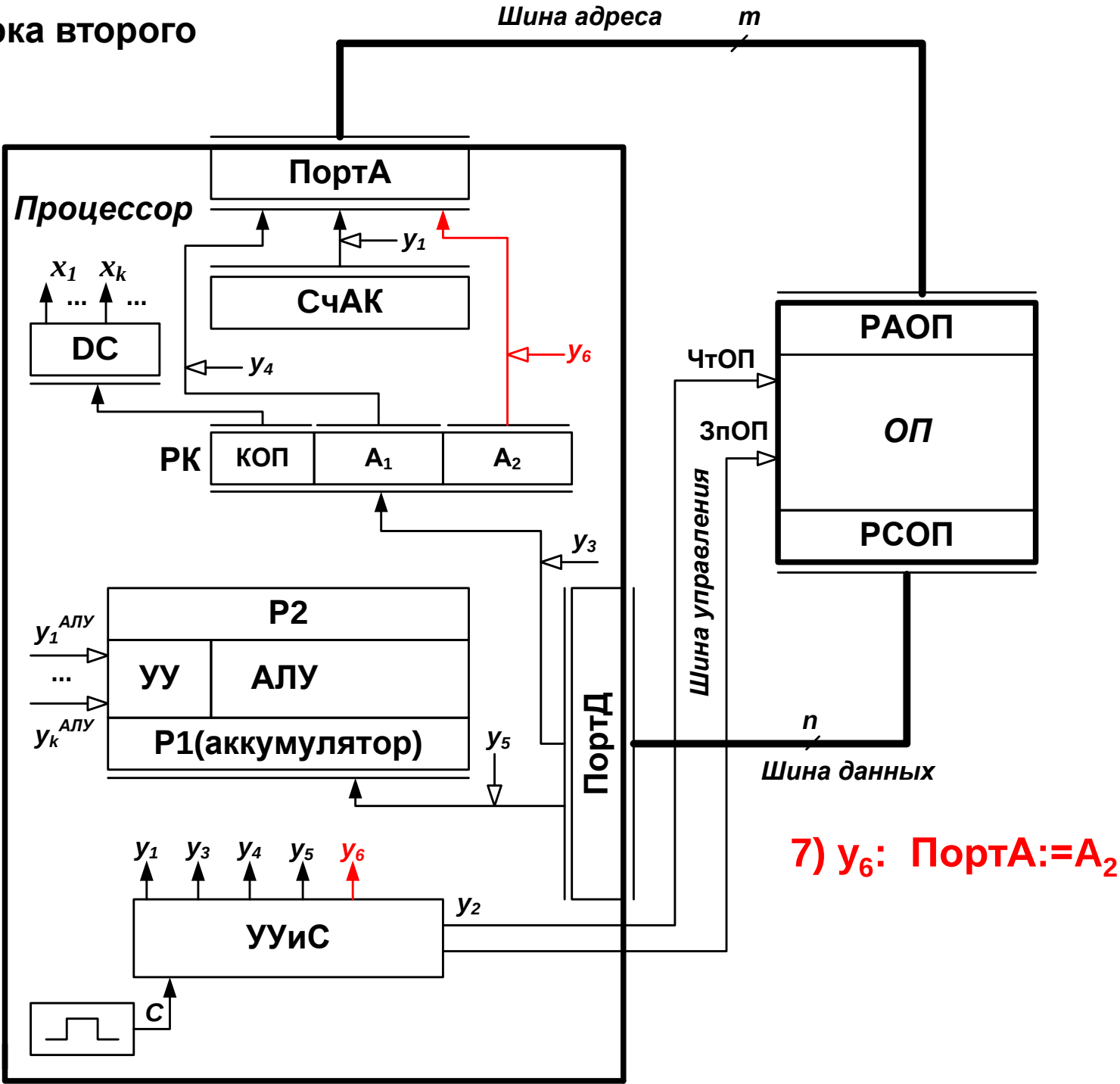
2.1. Выборка первого операнда



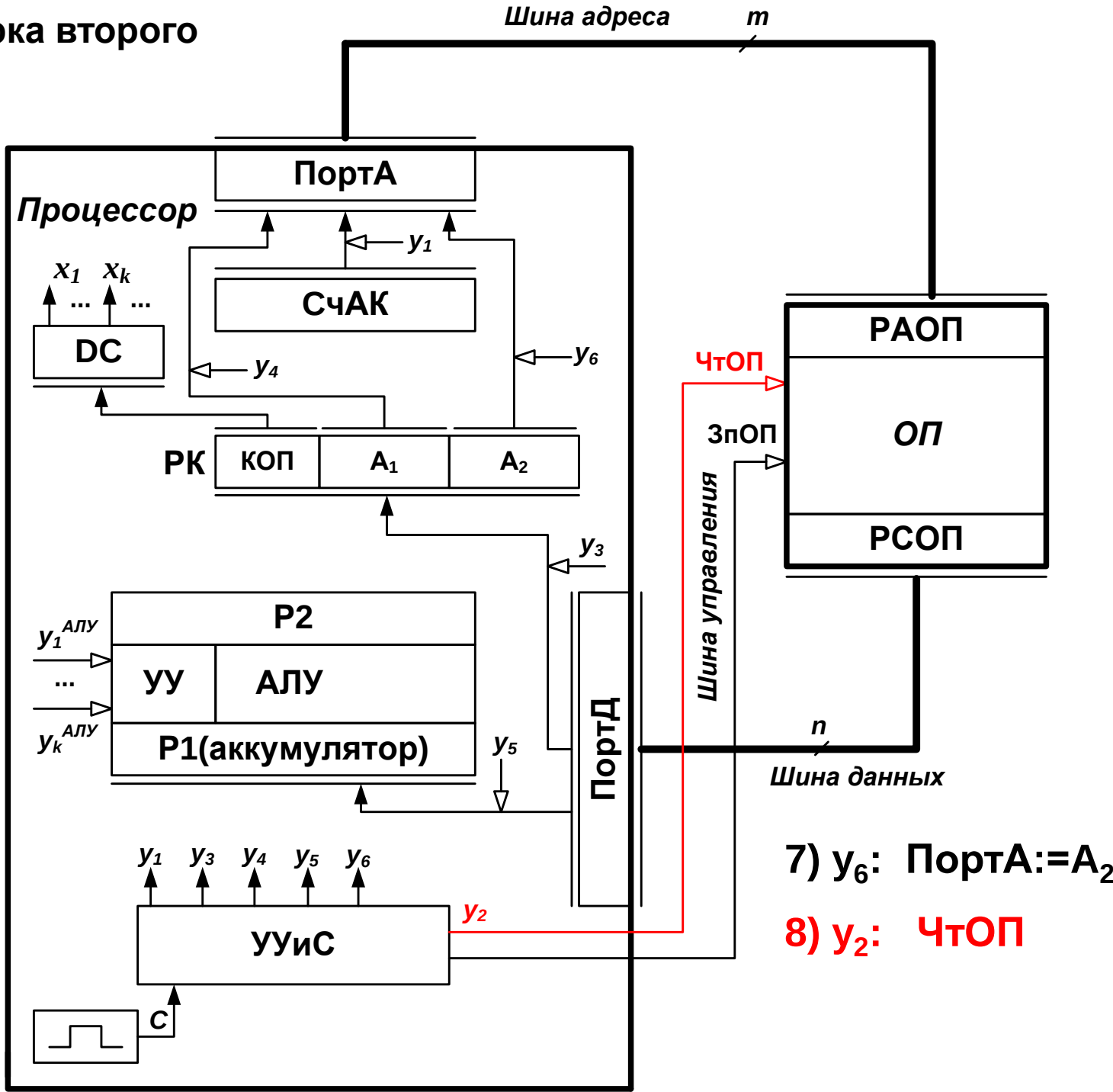
2.1. Выборка первого операнда



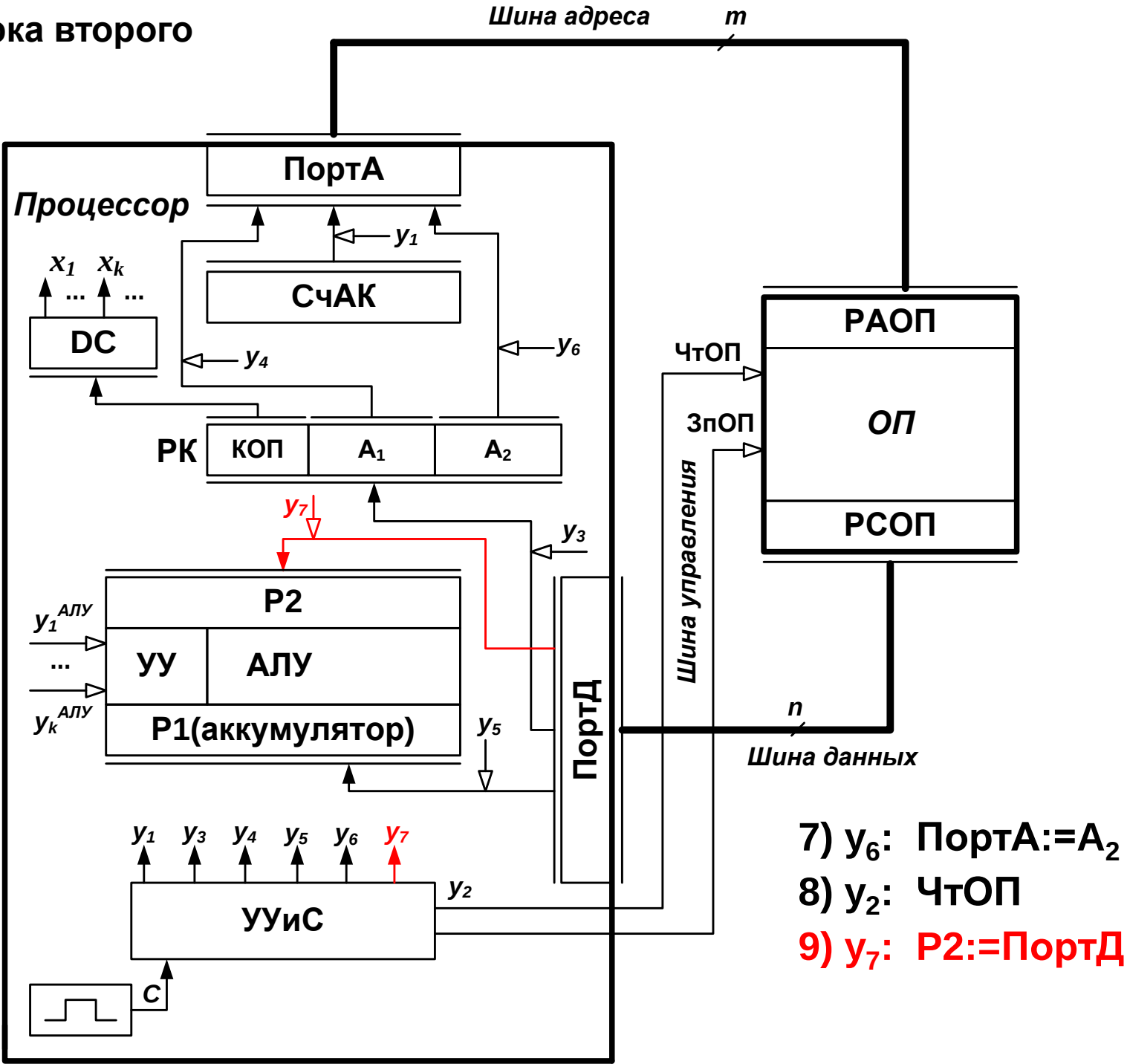
2.2. Выборка второго операнда



2.2. Выборка второго операнда



2.2. Выборка второго операнда



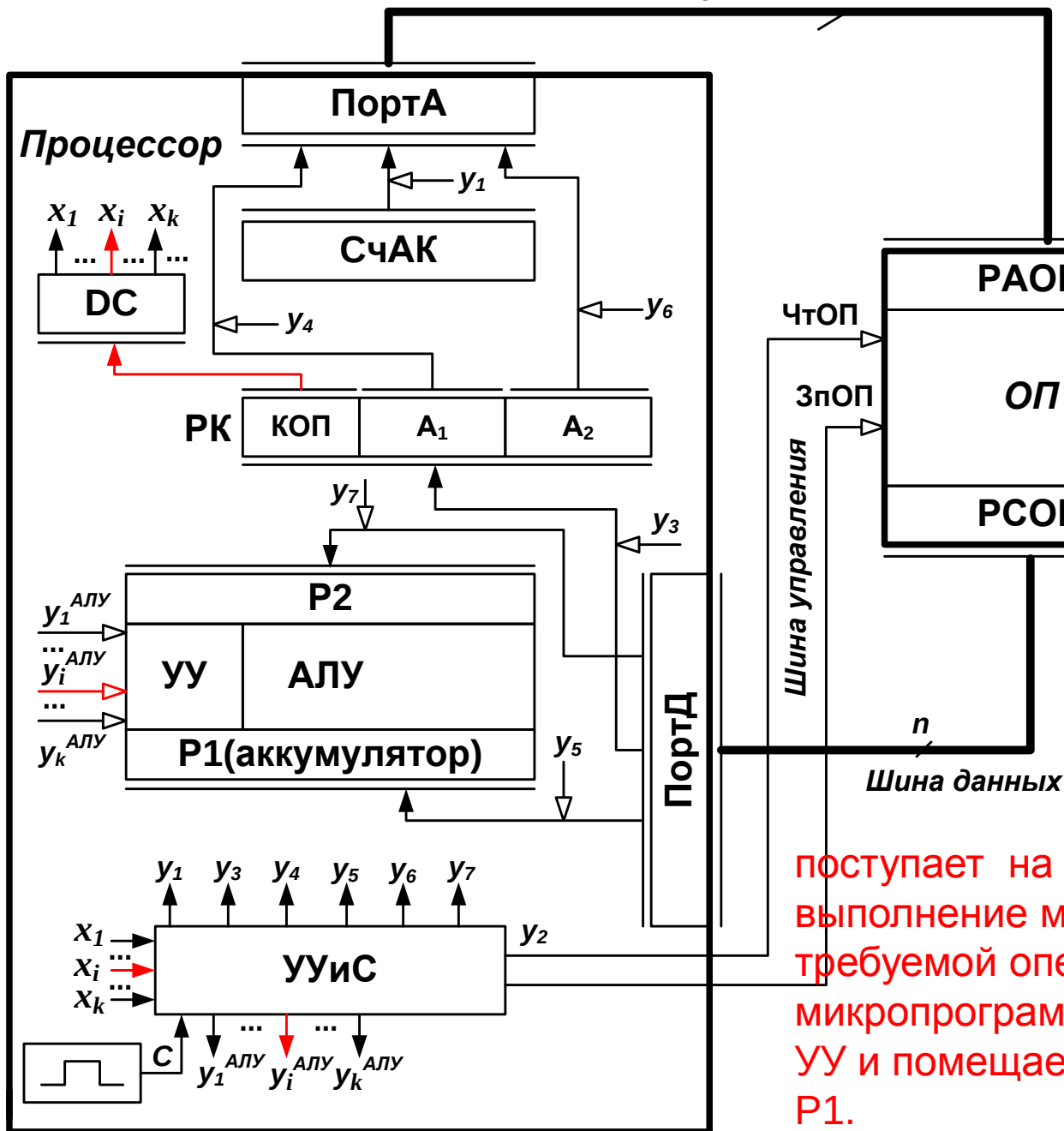
Рабочий цикл процессора

3 этап.

**Выполнение операции в
АЛУ**

10) $KOP \rightarrow DC \rightarrow x_i \rightarrow УУиС \rightarrow y_i \rightarrow АЛУ$

Шина адреса m

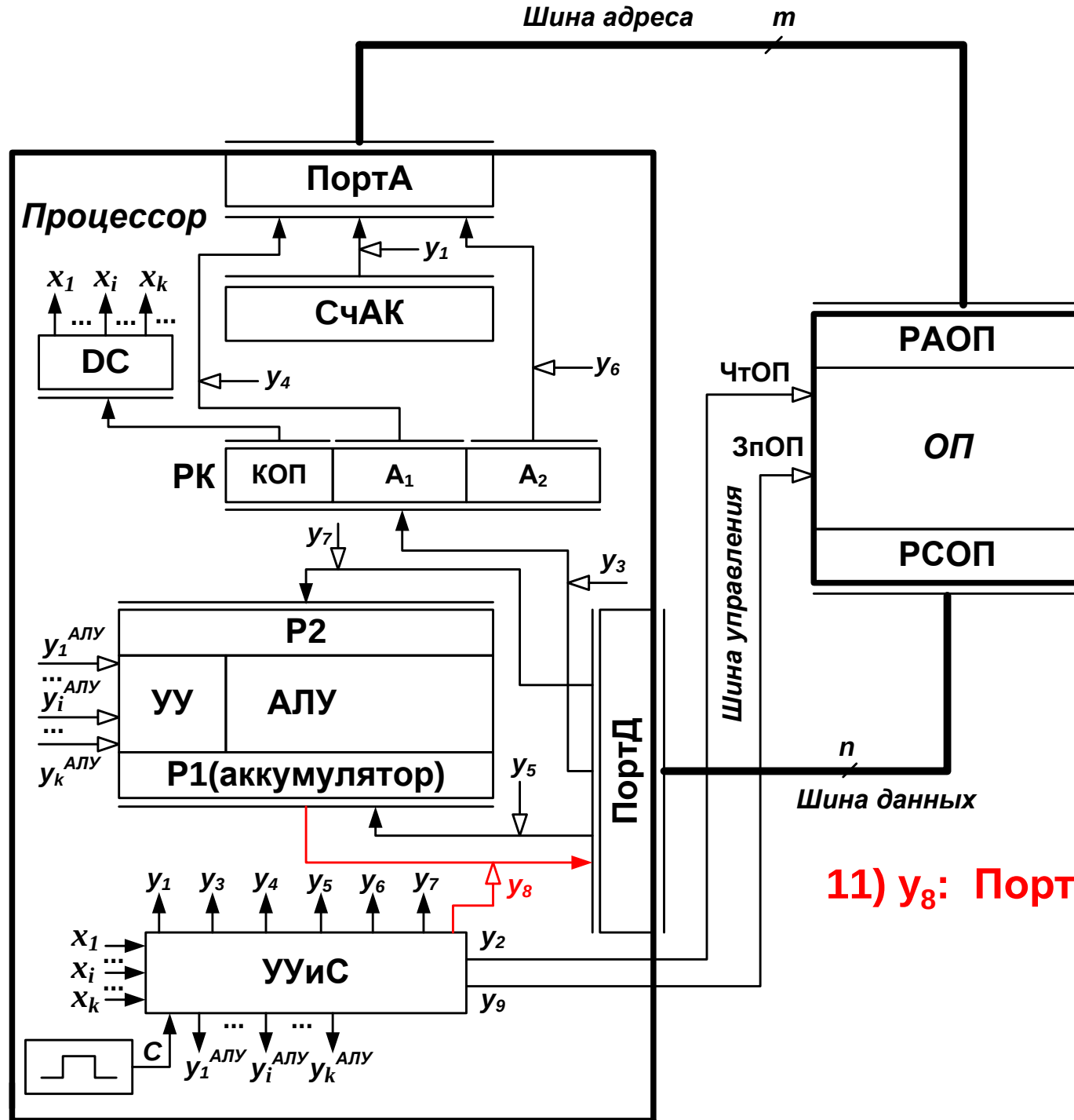


Значение в части РК, соответствующей КОП, расшифровывается на DC. На соответствующем i -ом выходе DC формируется осведомительный сигнал $x_i=1$. УУиС, проанализировав сигналы на своих входах x_1-x_k , в ответ на входной сигнал $x_i=1$ выдает выходной сигнал $y_i^{АЛУ}=1$. Этот управляющий сигнал поступает на АЛУ, в котором инициирует выполнение микропрограммы для требуемой операции. АЛУ выполняет микропрограмму под управлением своего УУ и помещает результат в аккумулятор Р1.

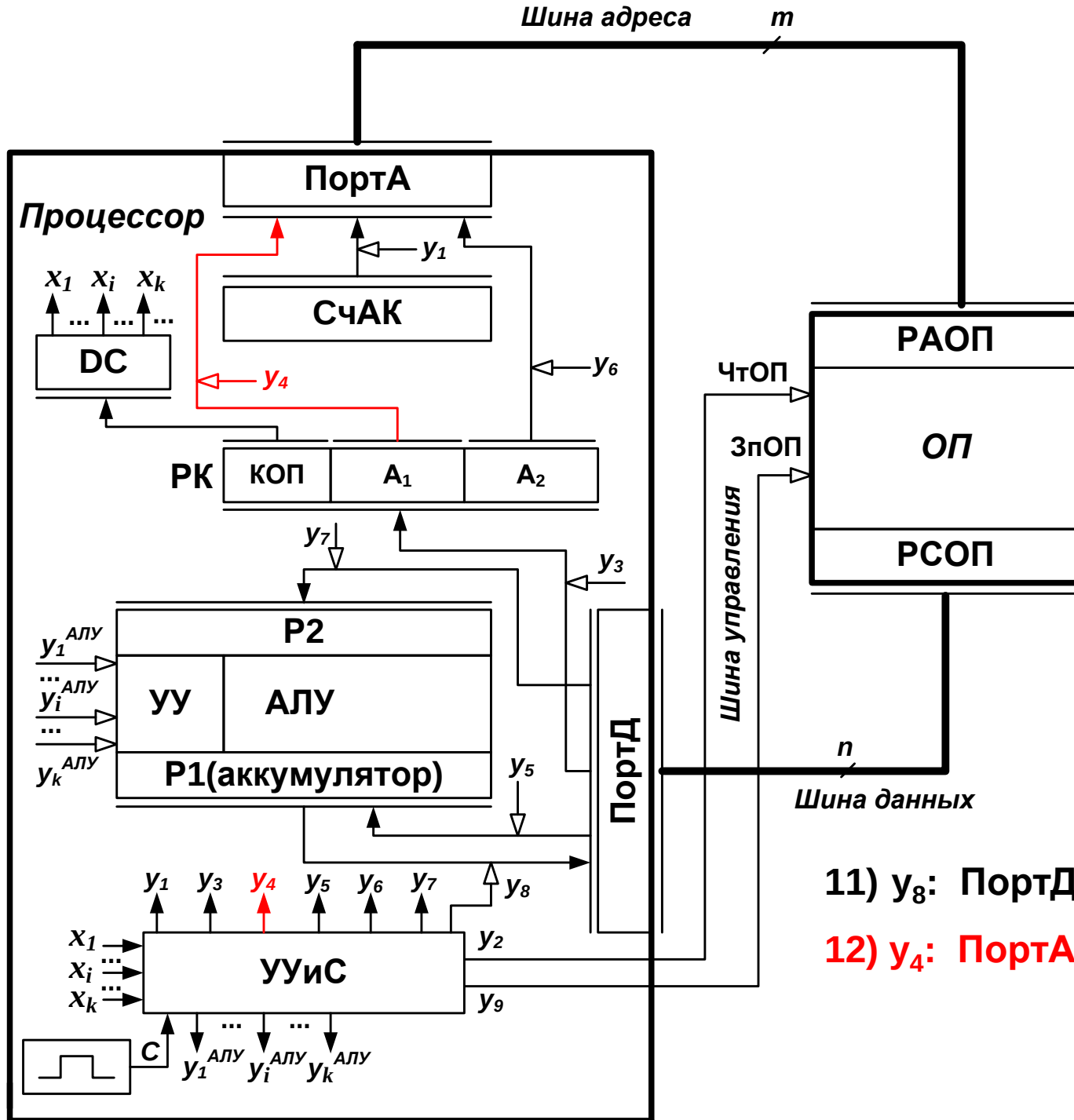
Рабочий цикл процессора

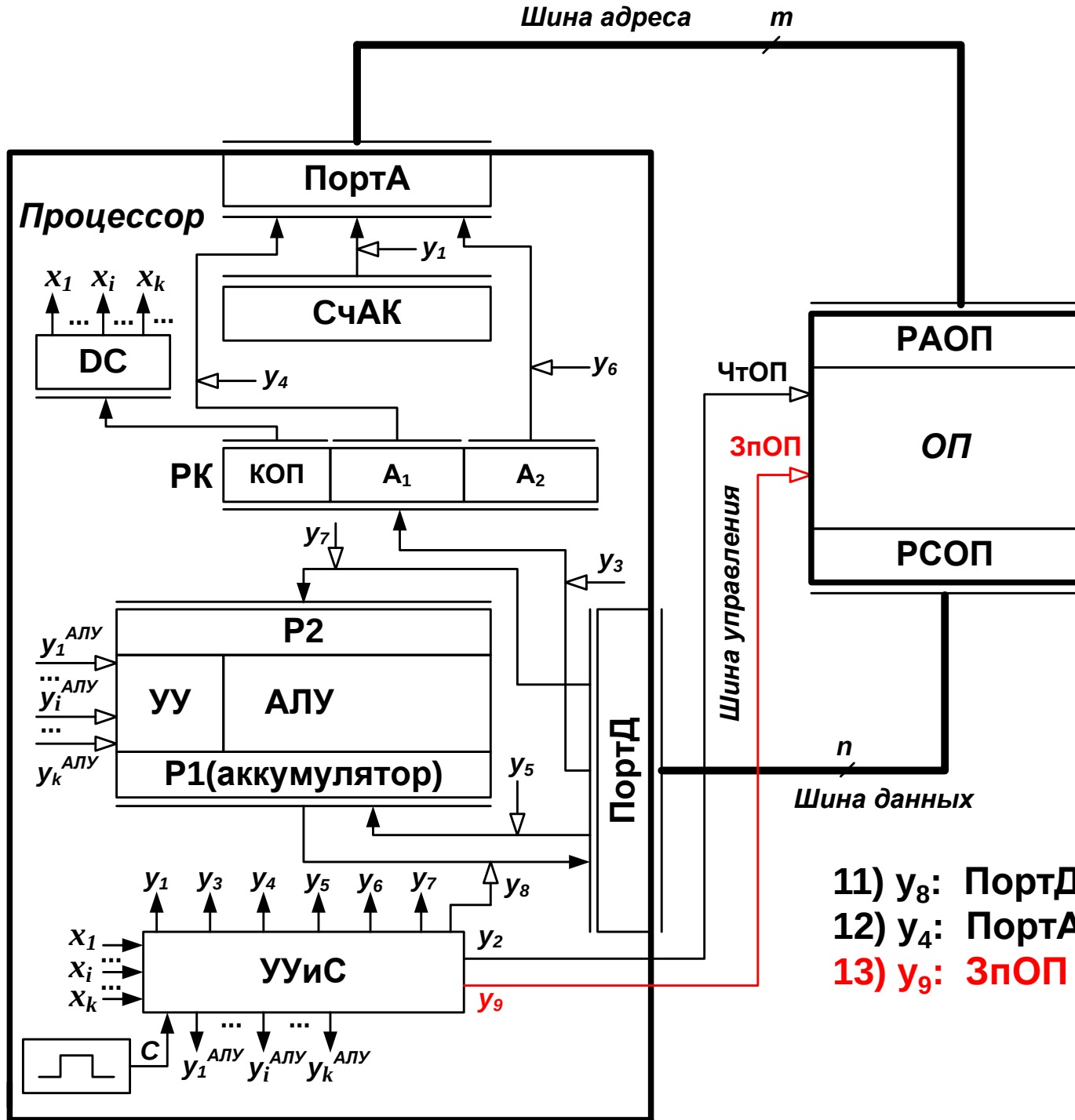
4 этап.

**Запись результата в ОП
(на место первого операнда)**



11) y_8 : ПортД:=Р1



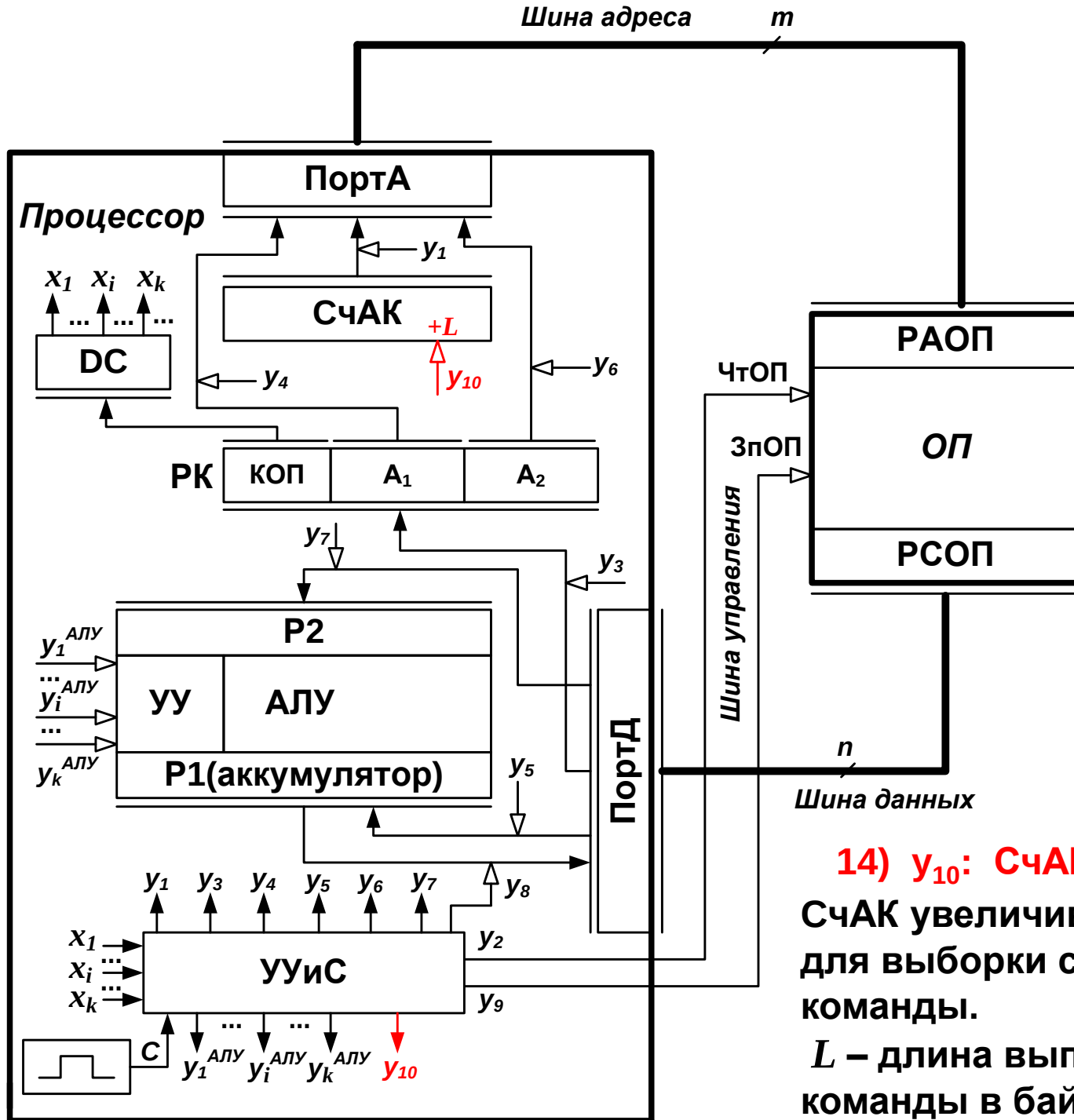


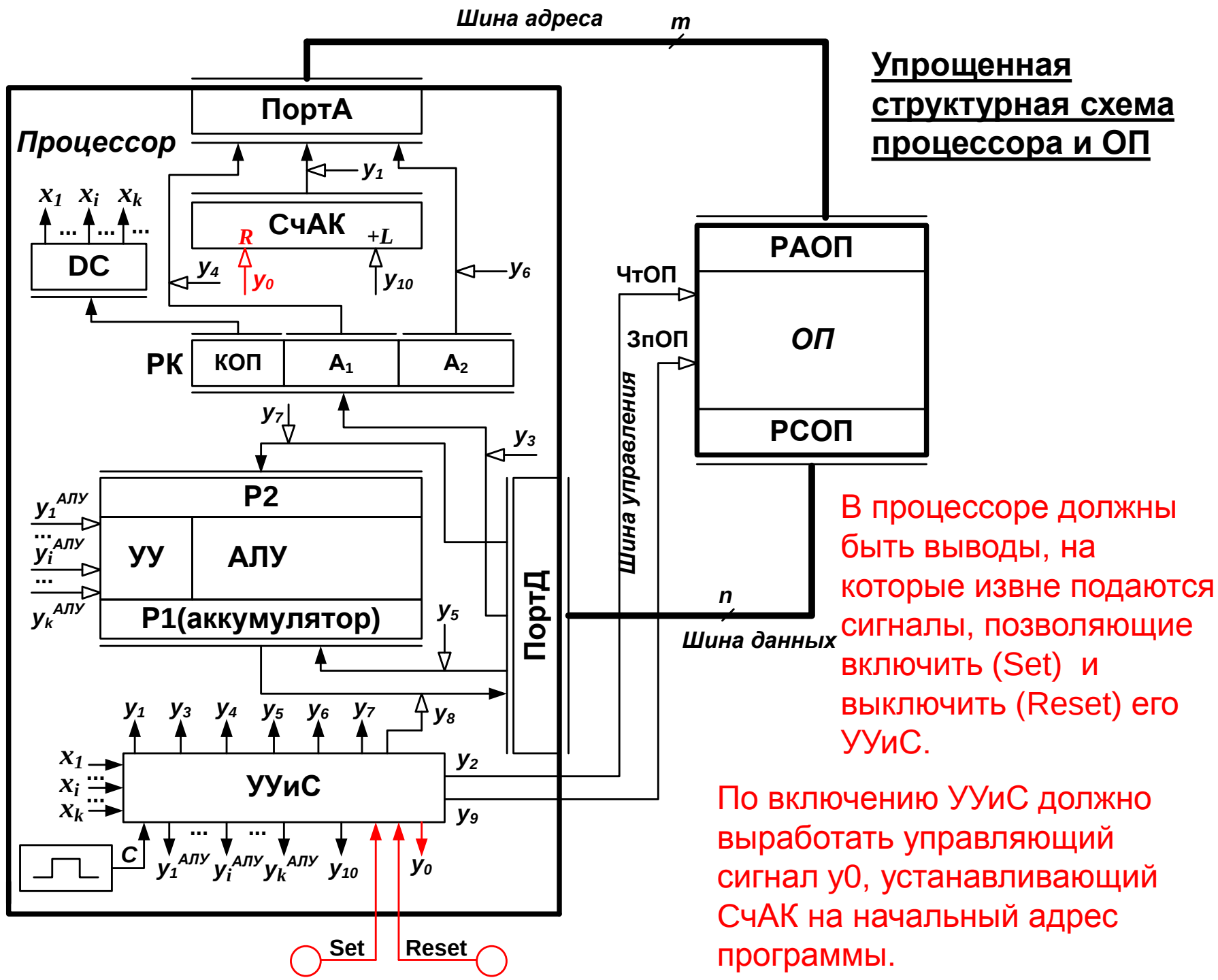
Рабочий цикл процессора

5 этап.

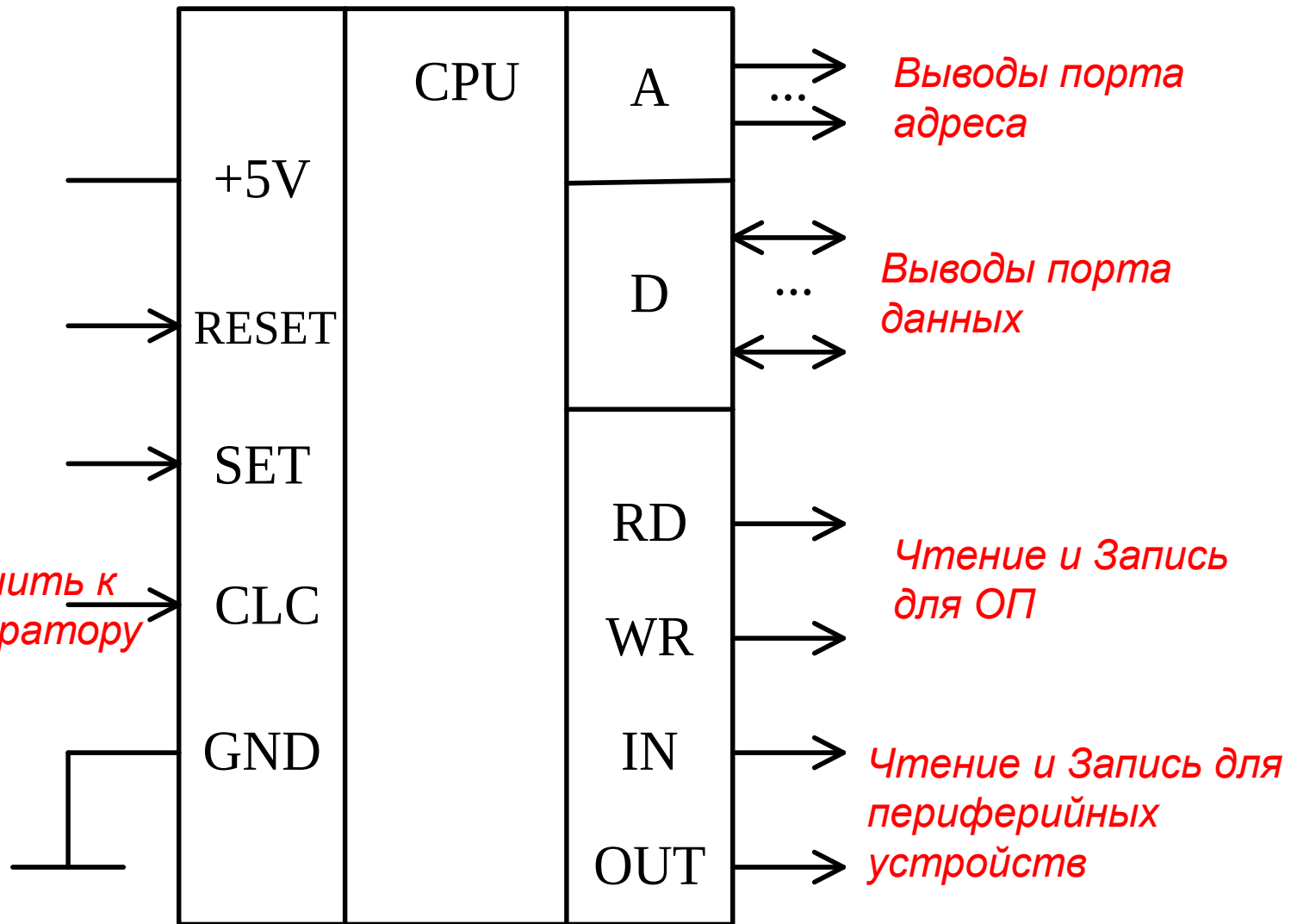
Продвижение СчАК

**(этот этап можно было выполнить сразу после этапа
выборки команды)**





Обобщенное условно-графическое обозначение (УГО) процессора



УУиС после включения реализует микропрограмму работы процессора, состоящую в повторении этапов его рабочего цикла:

Исполнение
предыдущих команд

$Y_0, \dots, Y_1, Y_2, Y_3, Y_4, Y_2, Y_5, Y_6,$

$Y_2, Y_7, Y_i^{AJTU}, Y_8, Y_4, Y_9, Y_{10}, \dots,$

$Y_1, Y_2, Y_3, \dots$

Исполнение следующих
команд

Микропрограмма
обработки прерываний,
если они возникли в
процессе выполнения
текущей команды
(рассмотрим позже).

Пример.

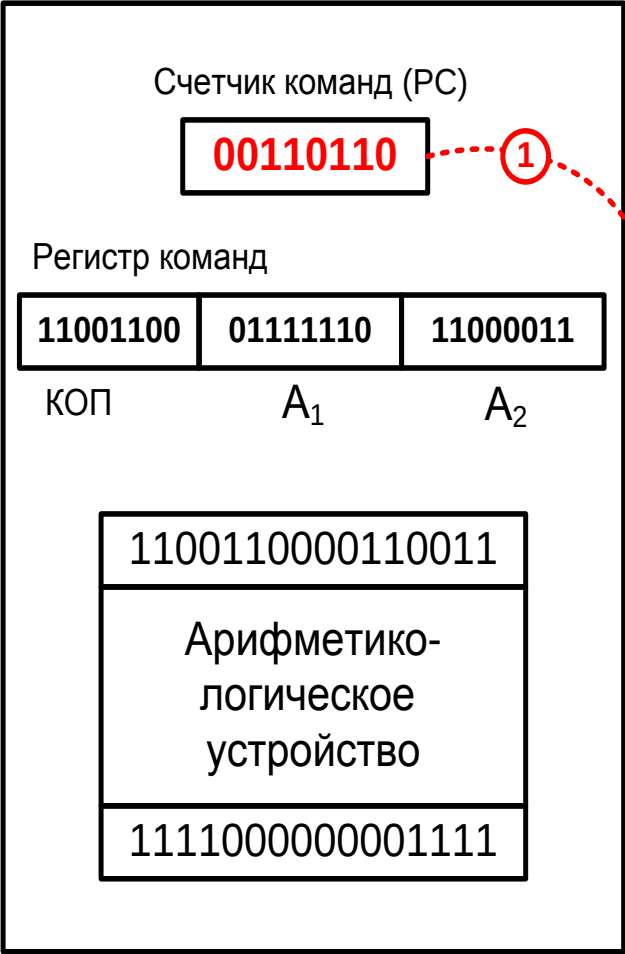
Проиллюстрируем последовательность изменения содержимого регистров процессора при выполнении арифметической команды (сложение чисел с фиксированной точкой).

Дано:

- 1) Система команд процессора включает в себя 256 команд (следовательно поле КОП должно содержать 8 бит).
- 2) В арифметической операции участвуют два операнда, находящиеся в ОП (в команде должно быть два поля для представления адресов операндов). Если длина операнда превышает 1 байт, то адресом операнда считается адрес его первого байта. Поле КОП однозначно определяет выполняемую операцию, а также тип (длину) операндов.
- 3) ОП организована как последовательность байтов, каждый байт имеет адрес (номер).
- 4) Ширина шины данных 32 бита (значит, из ОП памяти за одну операцию чтения (записи) может быть считано (записано) минимум 1 байт, максимум 4 байта).
- 5) Ширина шины адреса 8 бит (значит объем оперативной памяти не может превышать $2^8=256$ байтов).
- 6) ОП хранит и данные и команды (принстонская архитектура ОП, предложенная Фон-Нейманом).

СЧАК часто обозначают СК (счетчик команд) или PC (program counter).

В рассматриваемый момент времени в СК – адрес текущей команды, в РК – предыдущая команда, в регистрах АЛУ операнд и результат предыдущей команды. Центральный процессор (CPU)



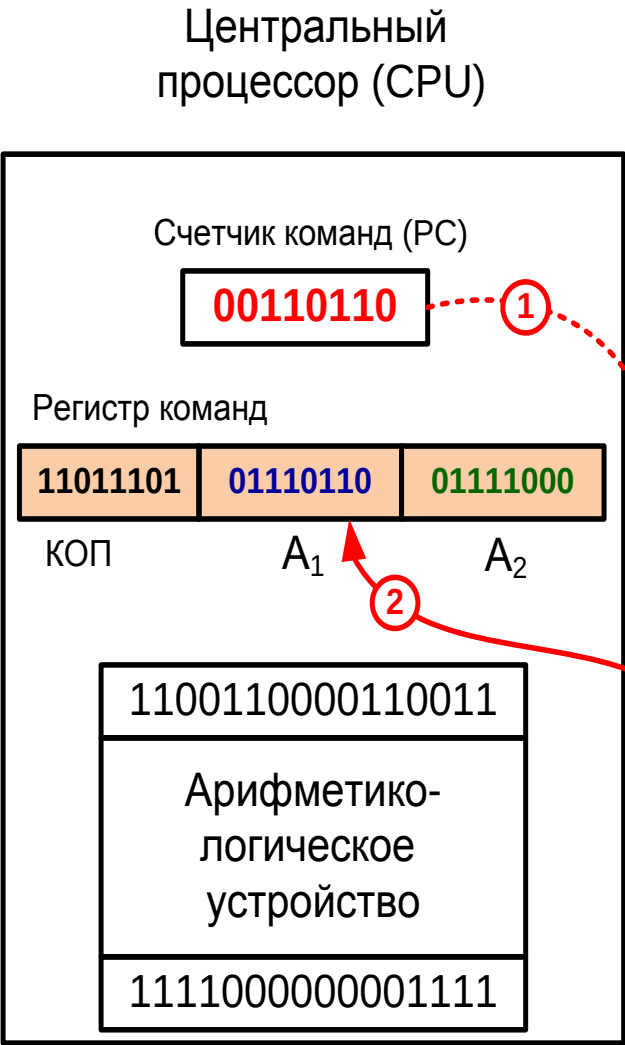
Оперативная память (Memory)

Адреса байтов	
11101110	11111111 (255)
...	
11101110	01111001 (121)
00110111	01111000 (120)
11010001	01110111 (119)
00010010	01110110 (118)
...	
11111000	00111001 (57)
01111000	00111000 (56)
01110110	00110111 (55)
11011101	00110110 (54)
...	
01011100	00000100 (4)
11110000	00000011 (3)
11000110	00000010 (2)
01001100	00000001 (1)
01001100	00000000 (0)

1) Процессор выставляет в порт адреса (на шину адреса) значение, находящееся в счетчике команд (СК).

Активные уровни управляющих сигналов чтения из ОП или записи в ОП выставляются процессором в нужные моменты времени на шине управления.

Оперативная память (Memory)



Адреса байтов

11101110	11111111 (255)
...	
11101110	01111001 (121)
00110111	01111000 (120)
11010001	01110111 (119)
00010010	01110110 (118)
...	
11111000	00111001 (57)
01111000	00111000 (56)
01110110	00110111 (55)
11011101	00110110 (54)
...	
01011100	00000100 (4)
11110000	00000011 (3)
11000110	00000010 (2)
01001100	00000001 (1)
01001100	00000000 (0)

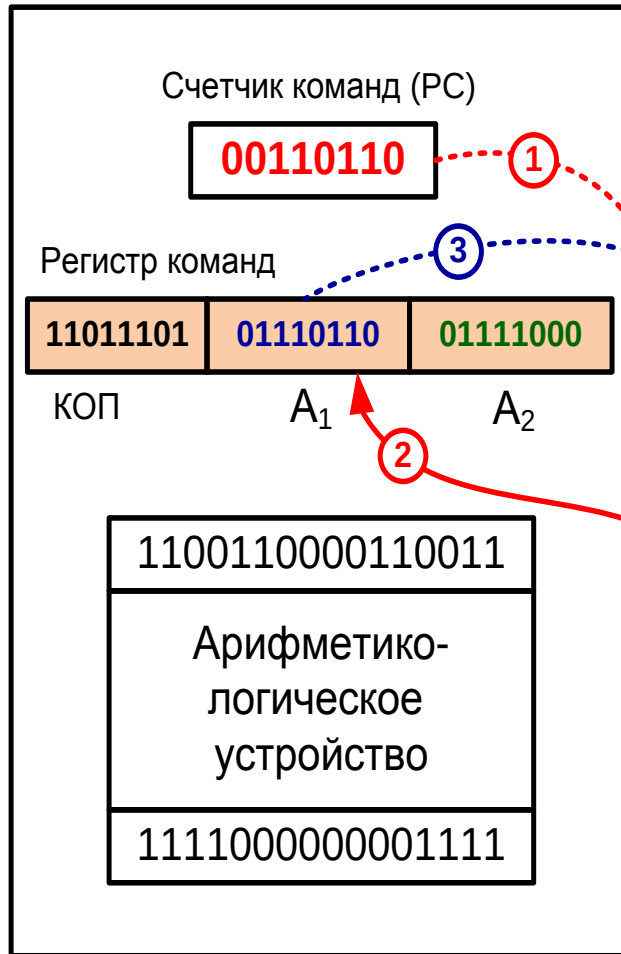
2) На шину данных (в порт данных) из ОП считывается очередная команда (адрес области ОП – на шине адреса). Информация с шины данных попадает в регистр команд (РК) процессора.

Оперативная память (Memory)

Адреса байтов

11101110	11111111 (255)
...	
11101110	01111001 (121)
00110111	01111000 (120)
11010001	01110111 (119)
00010010	01110110 (118)
...	
11111000	00111001 (57)
01111000	00111000 (56)
01110110	00110111 (55)
11011101	00110110 (54)
...	
01011100	00000100 (4)
11110000	00000011 (3)
11000110	00000010 (2)
01001100	00000001 (1)
01001100	00000000 (0)

Центральный процессор (CPU)



**3) В порт адреса (на шину адреса) из РК
выставляется адрес первого операнда.**

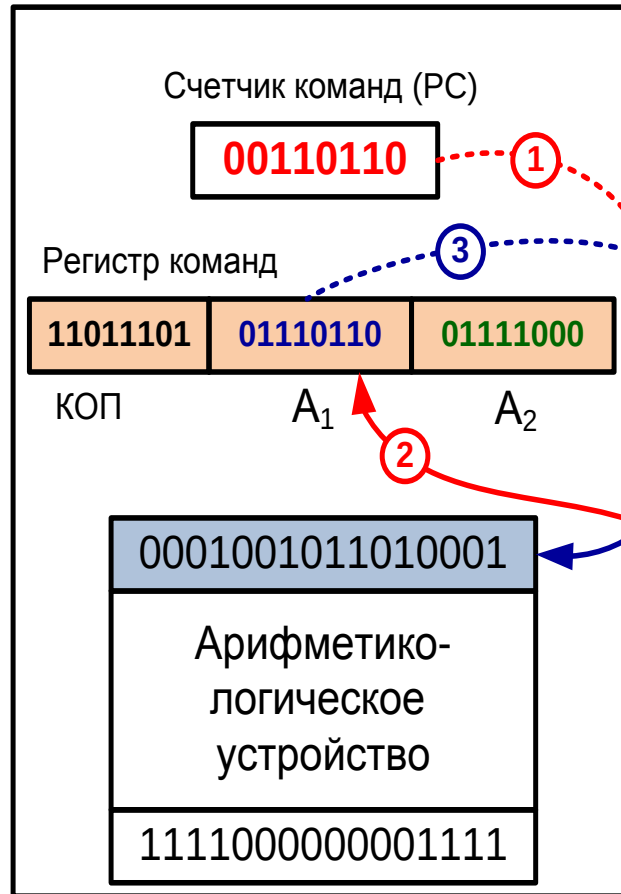
Оперативная память (Memory)

Адреса байтов

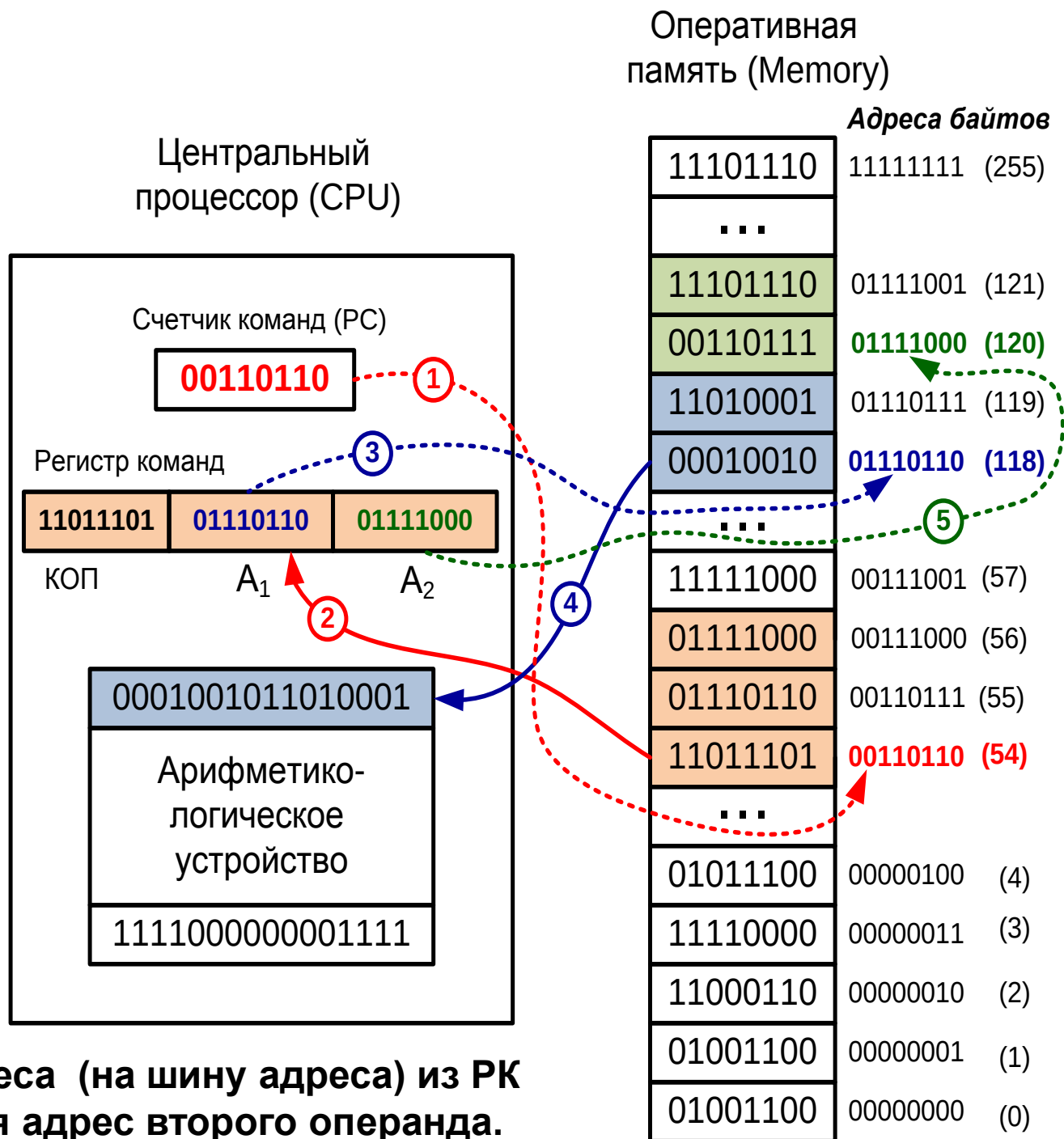
11101110	11111111 (255)
...	
11101110	01111001 (121)
00110111	01111000 (120)
11010001	01110111 (119)
00010010	01110110 (118)
...	
11111000	00110101 (57)
01111000	00111000 (56)
01110110	00110111 (55)
11011101	00110110 (54)
...	
01011100	00000100 (4)
11110000	00000011 (3)
11000110	00000010 (2)
01001100	00000001 (1)
01001100	00000000 (0)

Допустим, значение поля КОП подразумевает целочисленные операнды длиной 2 байта

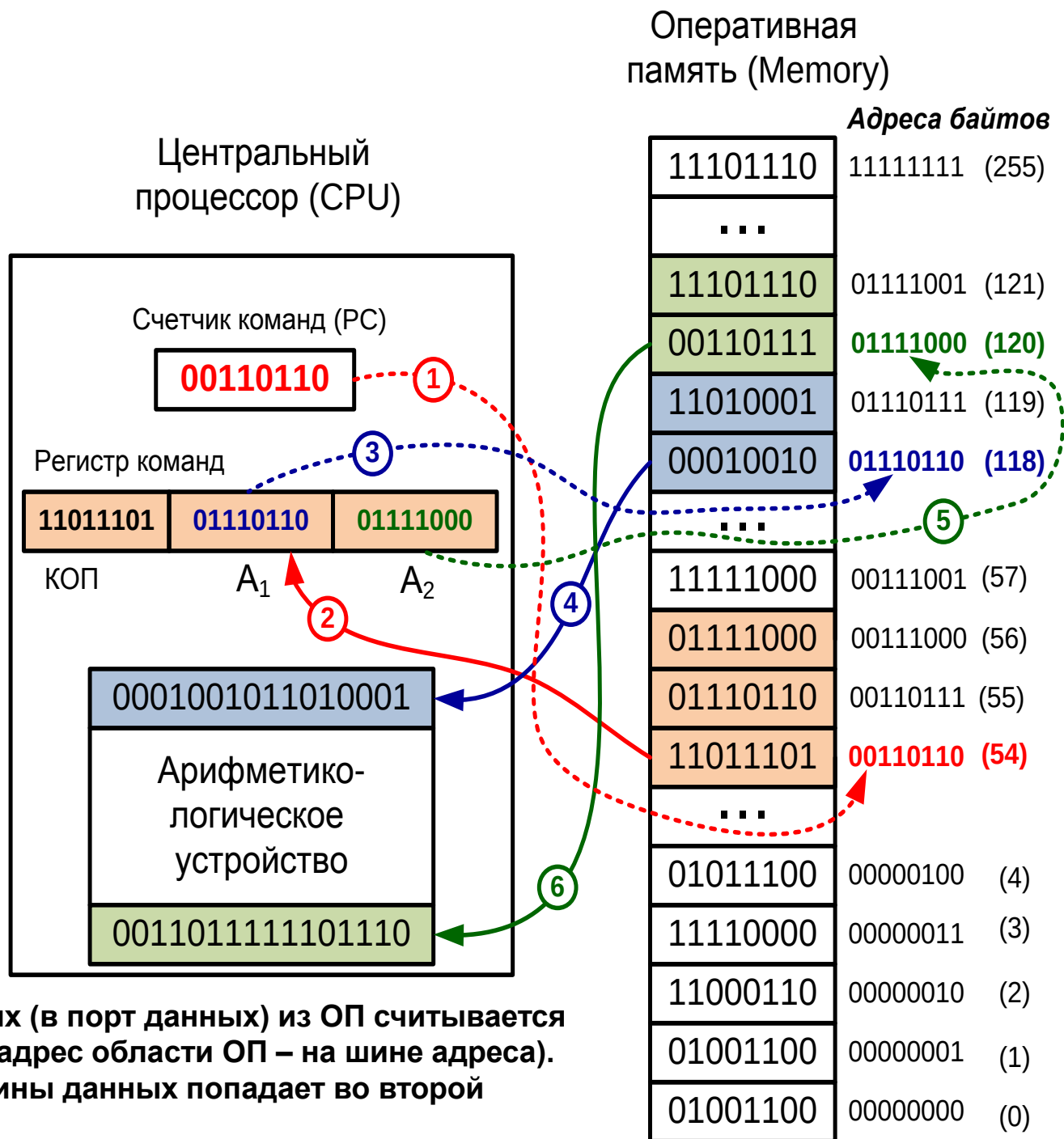
Центральный процессор (CPU)



4) На шину данных (в порт данных) из ОП считывается первый операнд (адрес области ОП – на шине адреса). Информация с шины данных попадает в первый регистр АЛУ (аккумулятор).



**5) В порт адреса (на шину адреса) из РК
выставляется адрес второго операнда.**



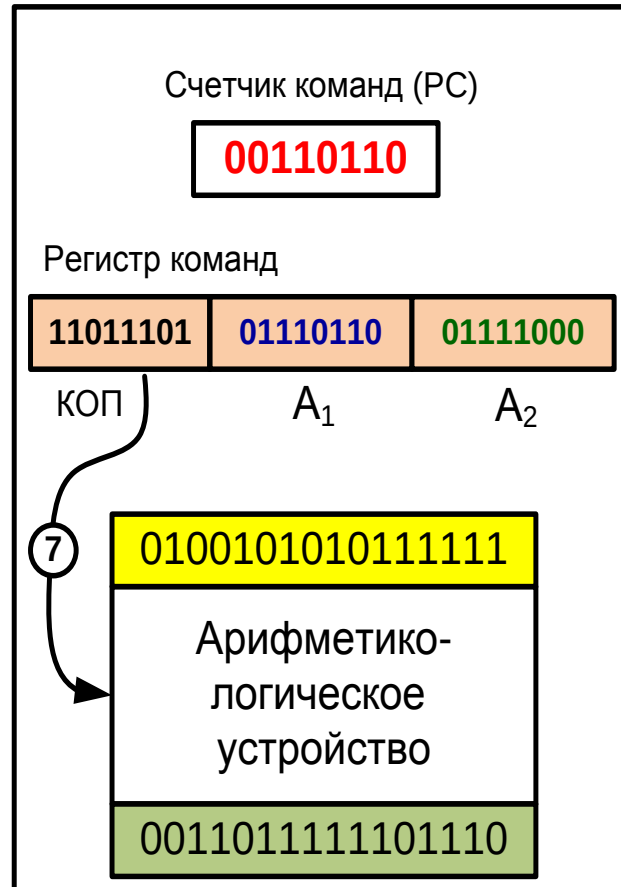
6) На шину данных (в порт данных) из ОП считывается второй операнд (адрес области ОП – на шине адреса). Информация с шины данных попадает во второй регистр АЛУ.

Оперативная память (Memory)

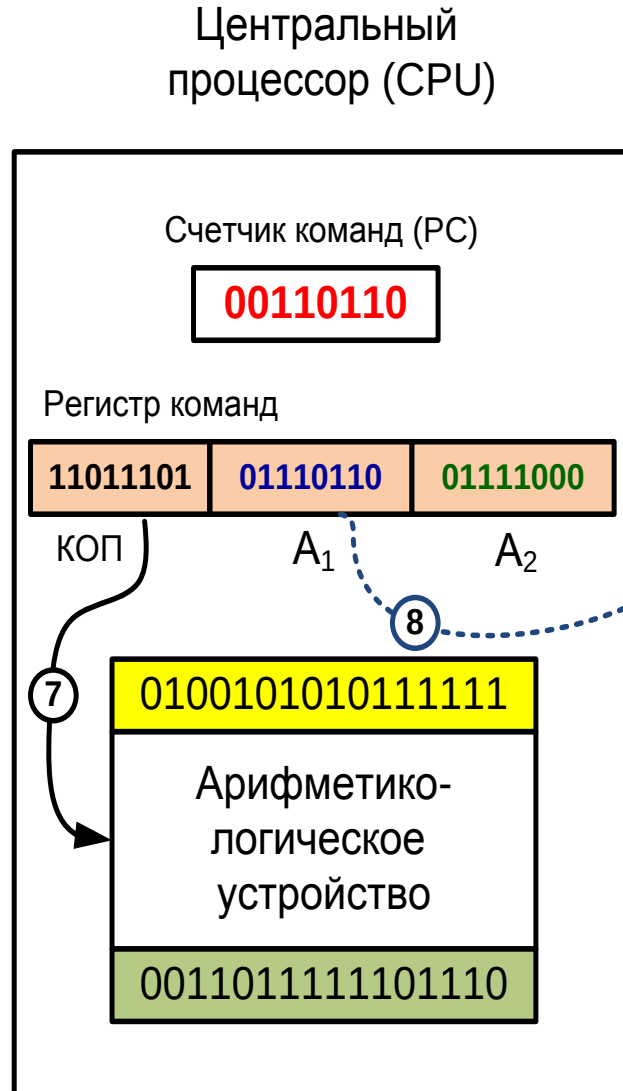
Адреса байтов

11101110	11111111 (255)
...	
11101110	01111001 (121)
00110111	01111000 (120)
11010001	01110111 (119)
00010010	01110110 (118)
...	
11111000	00111001 (57)
01111000	00111000 (56)
01110110	00110111 (55)
11011101	00110110 (54)
...	
01011100	00000100 (4)
11110000	00000011 (3)
11000110	00000010 (2)
01001100	00000001 (1)
01001100	00000000 (0)

Центральный процессор (CPU)



7) В АЛУ выполняется операция, соответствующая значению в поле КОП РК (здесь – сложение с фиксированной точкой). В аккумуляторе (выделен желтым цветом) появляется результат.



Оперативная память (Memory)

Адреса байтов	
11101110	11111111 (255)
...	
11101110	01111001 (121)
00110111	01111000 (120)
11010001	01110111 (119)
00010010	01110110 (118)
...	
11111000	00111001 (57)
01111000	00111000 (56)
01110110	00110111 (55)
11011101	00110110 (54)
...	
01011100	00000100 (4)
11110000	00000011 (3)
11000110	00000010 (2)
01001100	00000001 (1)
01001100	00000000 (0)

8) В порт адреса (на шину адреса) из РК выставляется адрес первого операнда.

Оперативная память (Memory)

Адреса байтов

11101110	11111111 (255)
...	
11101110	01111001 (121)
00110111	01111000 (120)
10111111	01110111 (119)
01001010	01110110 (118)
...	
11111000	00111001 (57)
01111000	00111000 (56)
01110110	00110111 (55)
11011101	00110110 (54)
...	
01011100	00000100 (4)
11110000	00000011 (3)
11000110	00000010 (2)
01001100	00000001 (1)
01001100	00000000 (0)

Центральный процессор (CPU)

Счетчик команд (PC)

00110110

Регистр команд

11011101

01110110

01111000

КОП

A₁

A₂

7

0100101010111111

8

9

Арифметико-логическое устройство

0011011111101110

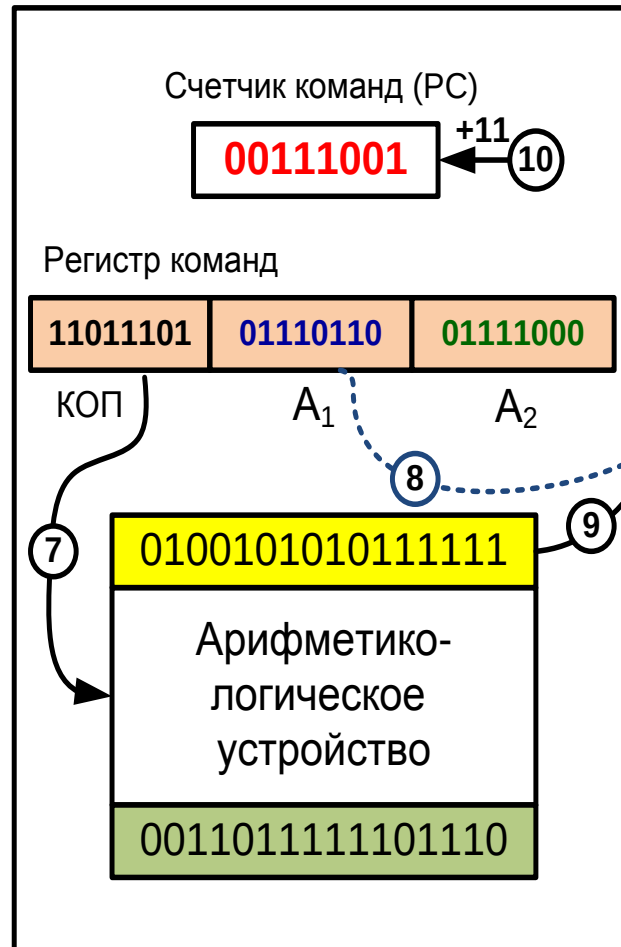
9) Результат из АЛУ по шине данных (через порт данных) записывается в ОП на место первого операнда (адрес области ОП – на шине адреса).

Оперативная память (Memory)

Адреса байтов

11101110	11111111 (255)
...	
11101110	01111001 (121)
00110111	01111000 (120)
10111111	01110111 (119)
01001010	01110110 (118)
...	
11111000	00111001 (57)
01111000	00111000 (56)
01110110	00110111 (55)
11011101	00110110 (54)
...	
01011100	00000100 (4)
11110000	00000011 (3)
11000110	00000010 (2)
01001100	00000001 (1)
01001100	00000000 (0)

Центральный процессор (CPU)



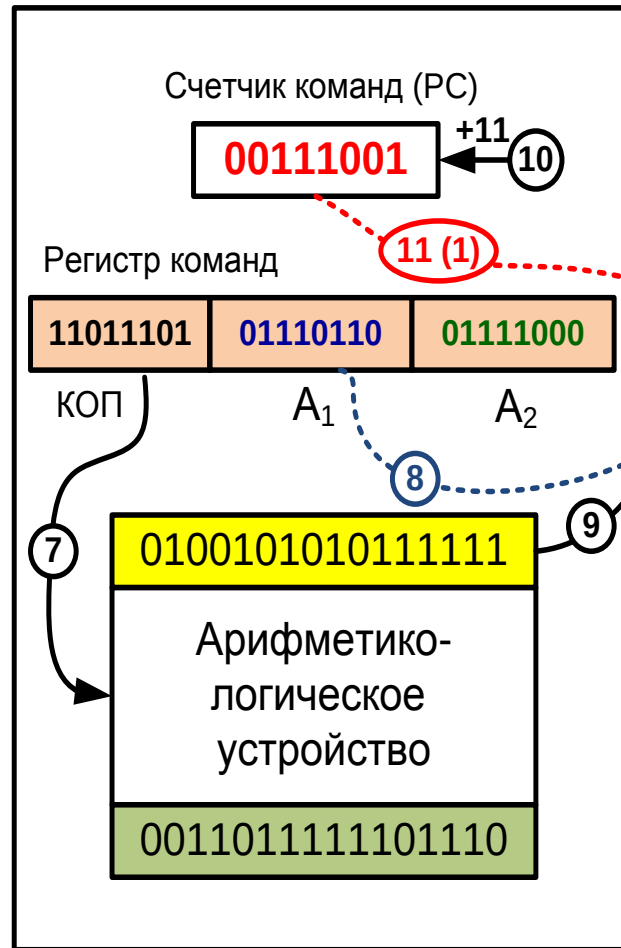
10). Значение СК увеличивается на длину выполненной команды (в примере она равна 3 байтам).

Оперативная память (Memory)

Адреса байтов

11101110	11111111 (255)
...	
11101110	01111001 (121)
00110111	01111000 (120)
10111111	01110111 (119)
01001010	01110110 (118)
...	
11111000	00111001 (57)
01111000	00111000 (56)
01110110	00110111 (55)
11011101	00110110 (54)
...	
01011100	00000100 (4)
11110000	00000011 (3)
11000110	00000010 (2)
01001100	00000001 (1)
01001100	00000000 (0)

Центральный процессор (CPU)



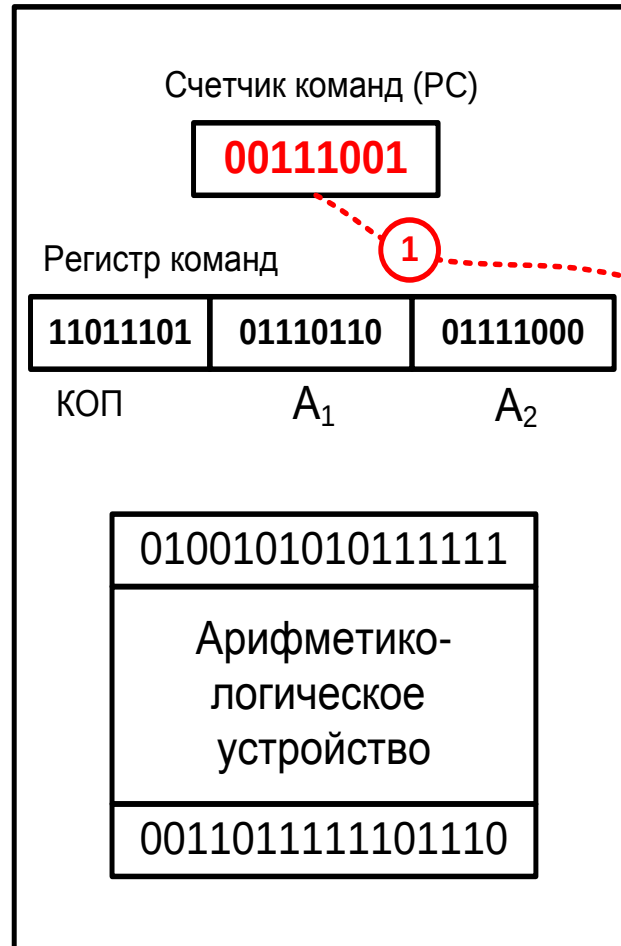
11(1). И весь процесс повторяется для следующей команды.

Оперативная память (Memory)

Адреса байтов

11101110	11111111 (255)
...	
11101110	01111001 (121)
00110111	01111000 (120)
10111111	01110111 (119)
01001010	01110110 (118)
...	
11111000	00111001 (57)
01111000	00111000 (56)
01110110	00110111 (55)
11011101	00110110 (54)
...	
01011100	00000100 (4)
11110000	00000011 (3)
11000110	00000010 (2)
01001100	00000001 (1)
01001100	00000000 (0)

Центральный процессор (CPU)



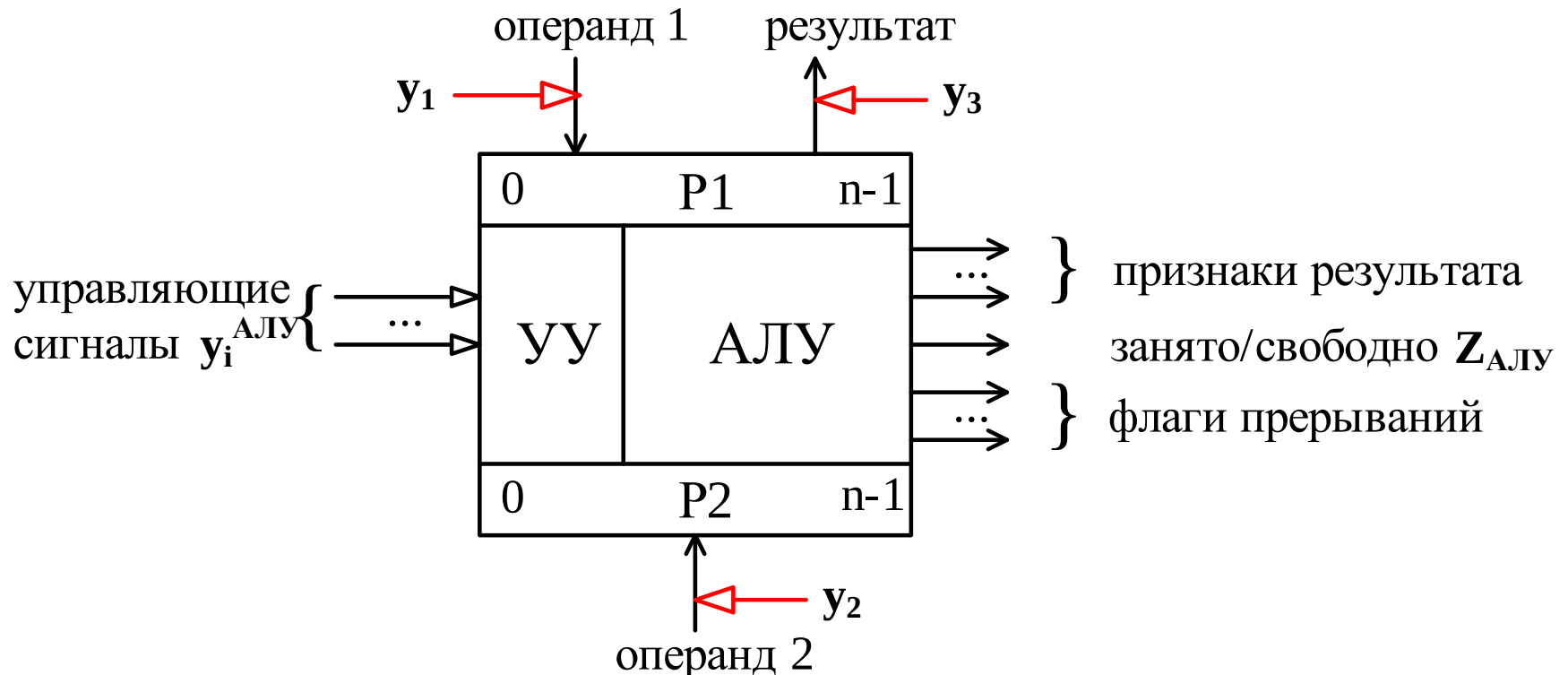
1) Процессор выставляет на шину адреса значение, находящееся в счетчике команд (СК)...

Вопросы, которые могут возникнуть:

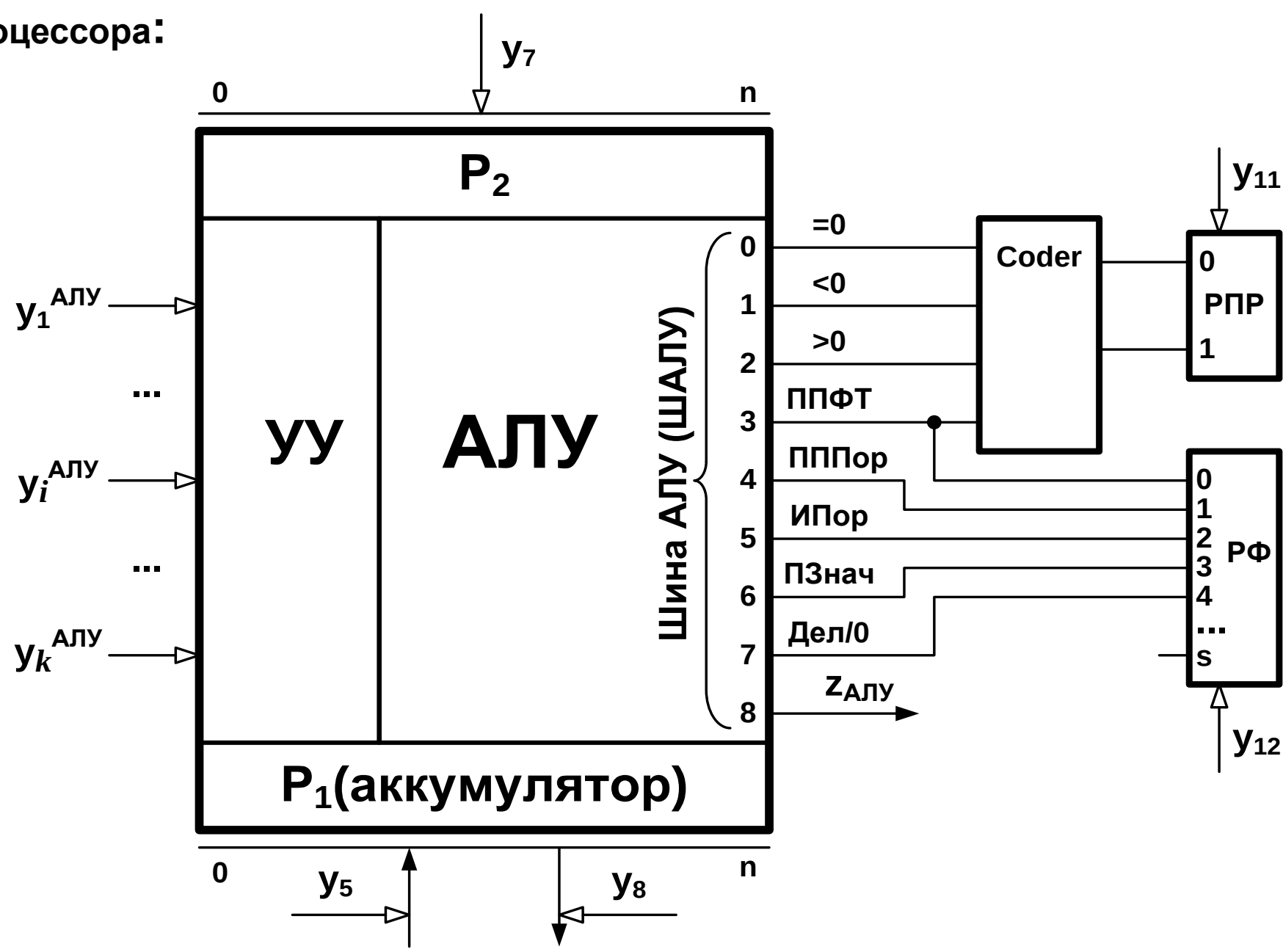
- 1) Как процессор узнает, что АЛУ закончило выполнение микропрограммы инициированной арифметической (или логической) операции?
- 2) Где и как сохраняется признак результата, который изменяют операции сложения, вычитания, инкрементирования, декрементирования и поразрядные логические операции ?

На рассмотренной выше структурной схеме процессора был изображен «облегченный» вариант автономного АЛУ.

В лекции, посвященной АЛУ, нами рассматривалась более полная схема:



Детализируем схему АЛУ для рассмотренной микропрограммы работы процессора:



1. Уточним 3 этап рабочего цикла процессора – «Выполнение операции в АЛУ». После того, как процессор загрузит в регистры АЛУ операнды (2 этап рабочего цикла) и инициирует выполнение нужной микропрограммы в АЛУ (3 этап рабочего цикла), АЛУ устанавливает на своем выходе $Z_{\text{АЛУ}}$ (ШАЛУ(8) – сигнал «занято/свободно») единицу (занято!). Процессор ожидает, пока АЛУ не установит ноль на выходе. Это будет означать, что микропрограмма исполнения операции завершилась, в Р1 находится результат, и теперь АЛУ готово для исполнения микропрограммы следующей операции (свободно!).

Помимо результата выполнения операции (в Р1) АЛУ формирует на выходной шине (ШАЛУ) унитарный код признака результата (только для операций сложения, вычитания, инкрементирования, декрементирования, и поразрядных логических операций: И, ИЛИ, Исключающее ИЛИ):

ШАЛУ(0)=1, если Р1=0;

ШАЛУ(1)=1, если Р1<0;

ШАЛУ(2)=1, если Р1>0;

ШАЛУ(3)=1, если произошла исключительная ситуация «Переполнение с фиксированной точкой».

Четырехразрядный унитарный код признака результата преобразуется с помощью шифратора (Coder) в двухразрядный позиционный код признака результата: (00 – результат равен нулю, 01 – результат меньше нуля, 10 – результат больше нуля, 11 – переполнение с фиксированной точкой).

Помимо признака результата АЛУ имеет на ШАЛУ линии, на которых формируются флаги возникших исключительных ситуаций при выполнении операций с плавающей точкой и операций деления:

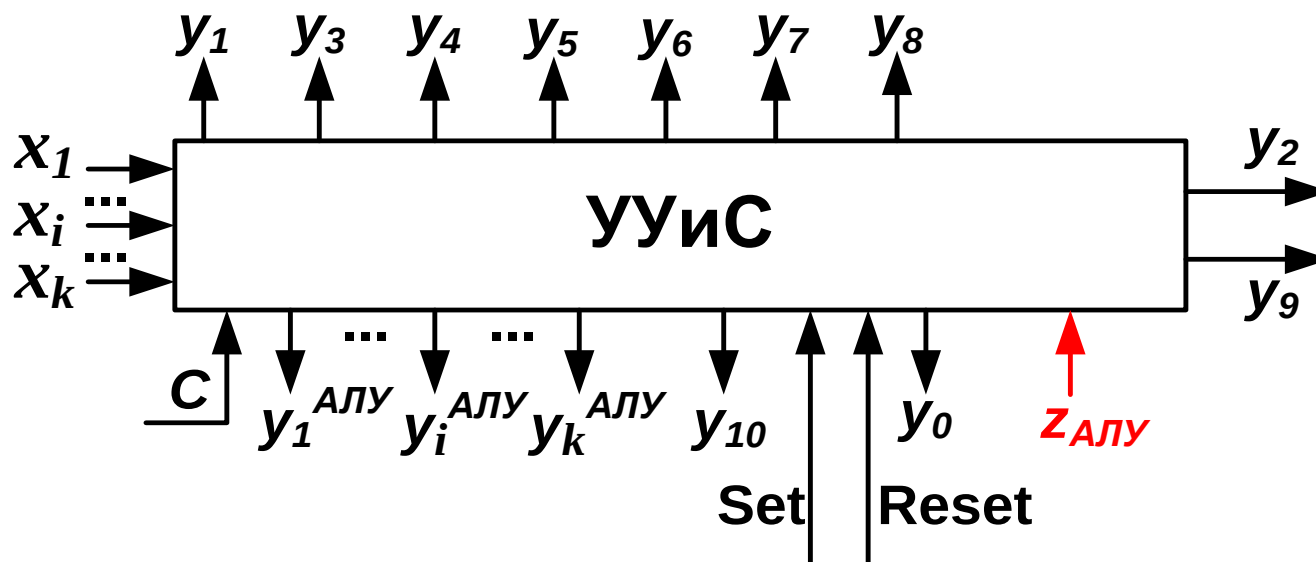
ШАЛУ(4)=1 – переполнение порядка;

ШАЛУ(5)=1 – исчезновение порядка;

ШАЛУ(6)=1 – потеря значимости;

ШАЛУ(8)=1 – деление на ноль.

После того как УУиС процессора получит от АЛУ осведомительный сигнал ШАЛУ(8)=0 ($Z_{\text{АЛУ}}=0$), процессор переходит к следующему (четвертому) этапу своего рабочего цикла – «Запись результата в ОП».



Уточним 4 этап рабочего цикла процессора – «Запись результата в ОП»

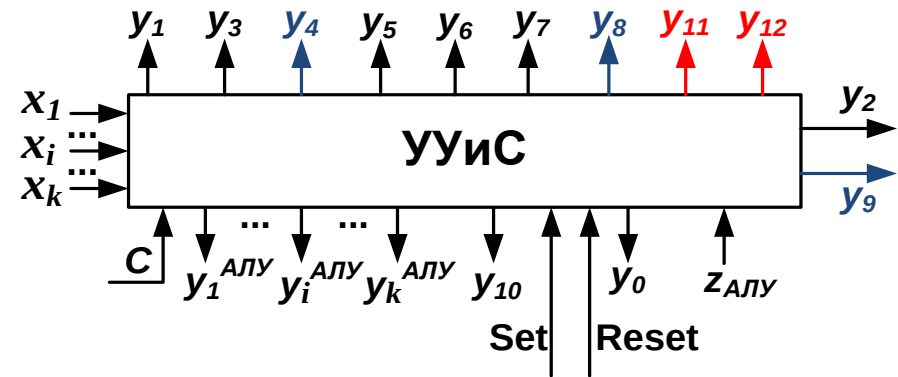
11.1) y_8 : ПортД:=Р1

11.2) y_{11} : РПР(0:1):=Coder(ШАЛУ(0:3))
(только для add, sub, inc, dec, and, or, xor)

11.3) y_{12} : РФ(0:4):=ШАЛУ(3:7)

12) y_4 : ПортА:=А1

13) y_9 : ЗпОП



Помимо результата выполнения операции, на этом этапе (если выполнялась команда сложения, вычитания, инкрементирования, декрементирования, поразрядного И, ИЛИ, исключающего ИЛИ) сохраняется в регистре признака результата (РПР) позиционный код признака результата, сформированный шифратором (Coder). **Другие команды не должны изменять содержимое РПР!**

Флаги исключительных ситуаций, которые могут возникать при выполнении арифметических операций, запоминаются на регистре флагов прерываний (РФ), для их последующего анализа и обработки системой прерываний.

Если ШАЛУ(3:7)=00000, значит исключительной ситуации при выполнении операции в АЛУ не возникло, т.е. операция была выполнена успешно.

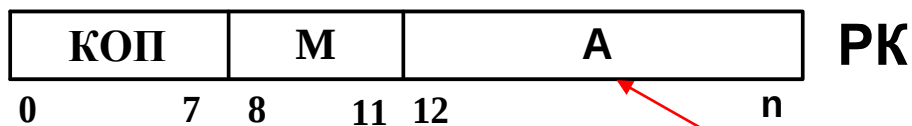
Внимание! В структуру процессора ввели еще два специальных регистра: двухразрядный РПР и (s+1)-разрядный РФ. Разрядность РФ больше числа флагов исключительных ситуаций на ШАЛУ, т.к. возможны прерывания не только от АЛУ, но и от таймеров, внешних устройств и т.п. (рассмотрим позже).

После выполнения арифметической (логической) команды счетчик команд всегда продвигается на длину выполненной команды. Таким образом, осуществляется переход к следующей по порядку команде.

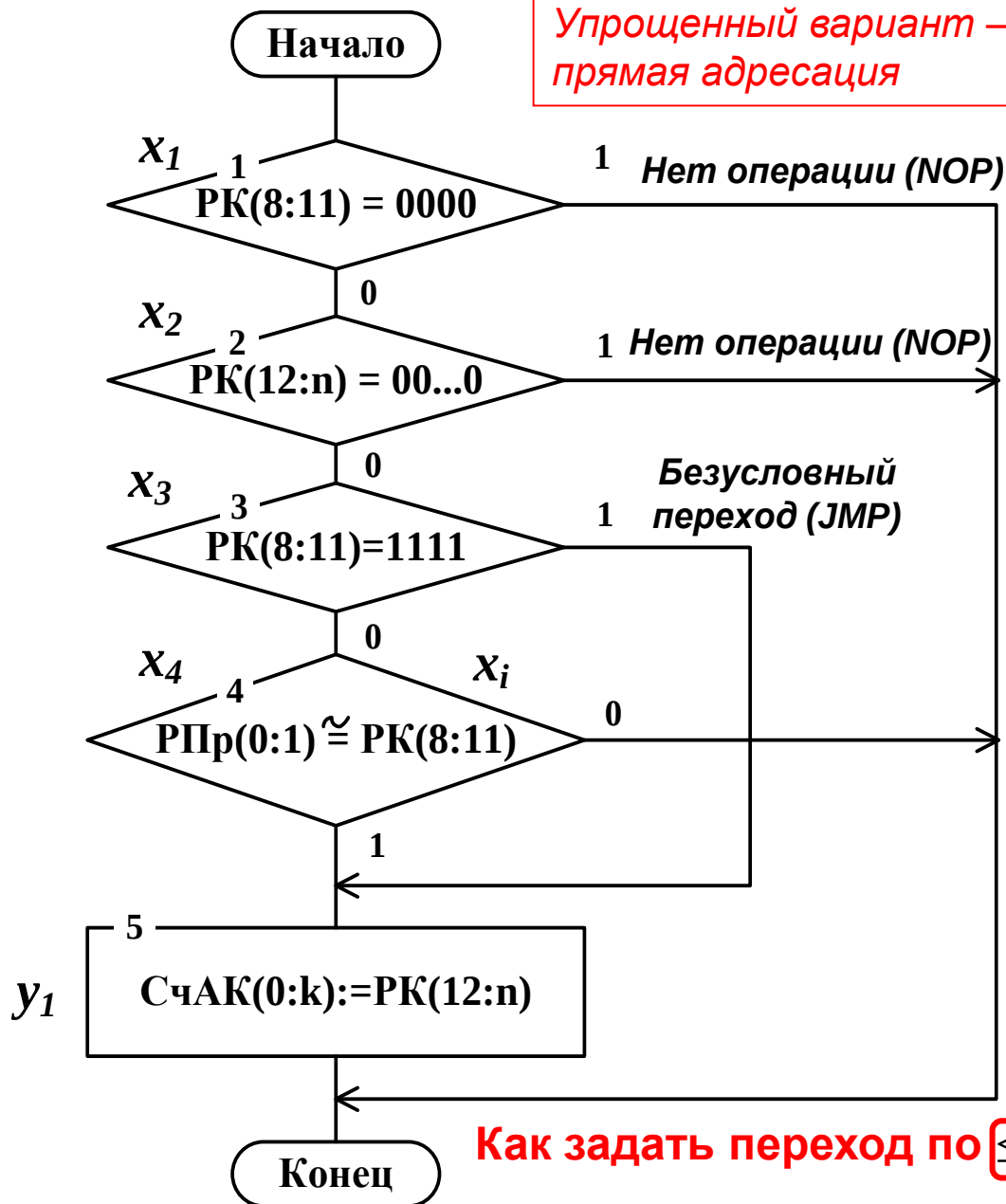
А как осуществить переход к произвольной команде, т.е. как организовать разветвляющийся или циклический вычислительный процесс?

Для этого служат команды перехода (передачи управления).

В качестве примера команды перехода рассмотрим команду условного перехода по маске.



Упрощенный вариант –
прямая адресация



Считаем, что первый этап рабочего цикла завершен, очередная команда выбрана на регистр команд, и это команда условного перехода по маске.

Формат команды:

КОП – РК(0:7) (до 256 команд в системе команд процессора);

М (маска) – РК(8:11);

Соответствие		Содержание
РПР	Разряд маски	
00	РК(8)	= 0
01	РК(9)	<0
10	РК(10)	>0
11	РК(11)	ППФТ

А – РК(12:n) – адрес перехода.

Описание команды.

Если в поле маски перехода М или в поле адреса перехода А нули, то переход не выполняется (рассматриваемая команда соответствует команде «нет операции»).

Если в поле М все единицы, переход выполняется обязательно (рассматриваемая команда соответствует команде безусловного перехода).

Если значение признака результата соответствует коду, указанному в поле М, продвинутый (СЧАК продвигается после выборки команды перехода перед ее выполнением) адрес команды в СЧАК замещается адресом перехода А. В противном случае продолжается выполнение обычной последовательности команд с использованием продвинутого адреса. Признак результата остается без изменения.

Прерывания программы отсутствуют.

Вопрос: Какова разрядность поля А (чему равно n) и какова разрядность СЧАК (чему равно k)?

Разрядность адреса в вычислительной системе (N_A) определяется в зависимости от объема ОП в байтах $E_{оп}$:

$$N_A = \log_2 E_{оп}$$

В частности, такую разрядность будут иметь СЧАК и поле А регистра команд (РК).

Фрагмент структурной схемы процессора, исполняющий команду условного перехода по маске.

Нумерация управляющих и осведомительных сигналов взята условно (не связана с нумерацией в рассмотренном ранее фрагменте, исполняющем арифметическую операцию).

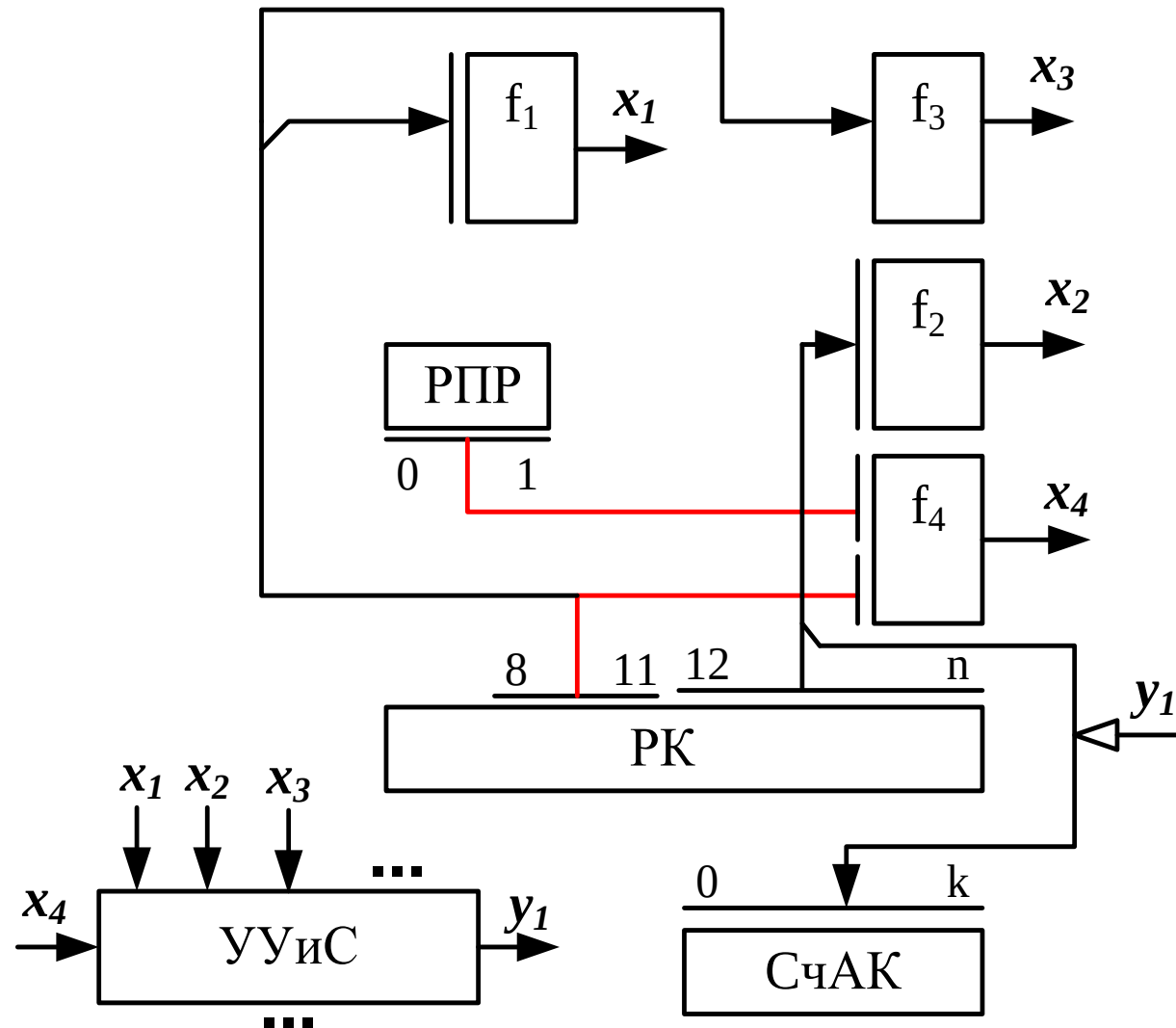
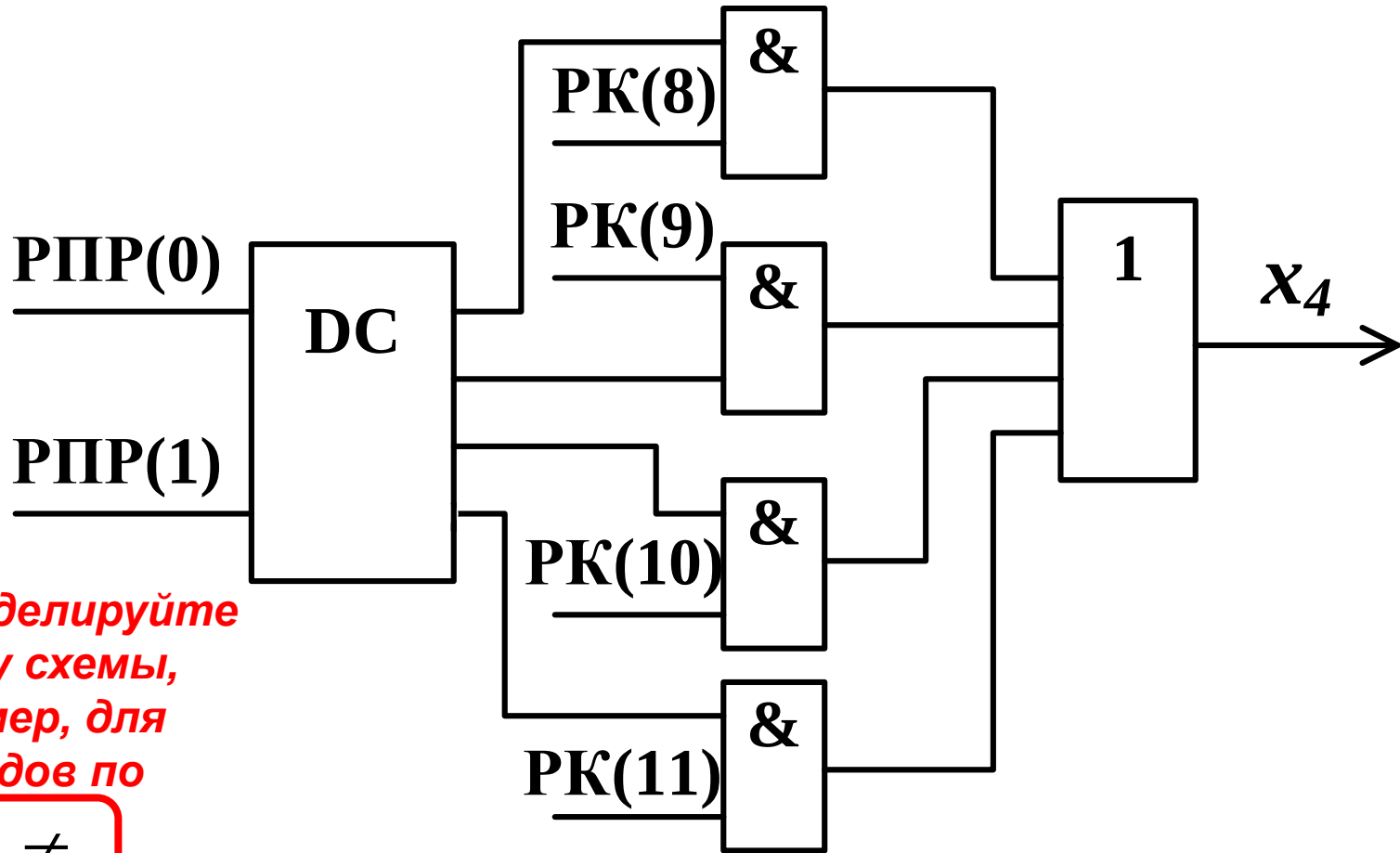


Схема вычисления осведомительного сигнала x_4 (булева функция f_4)



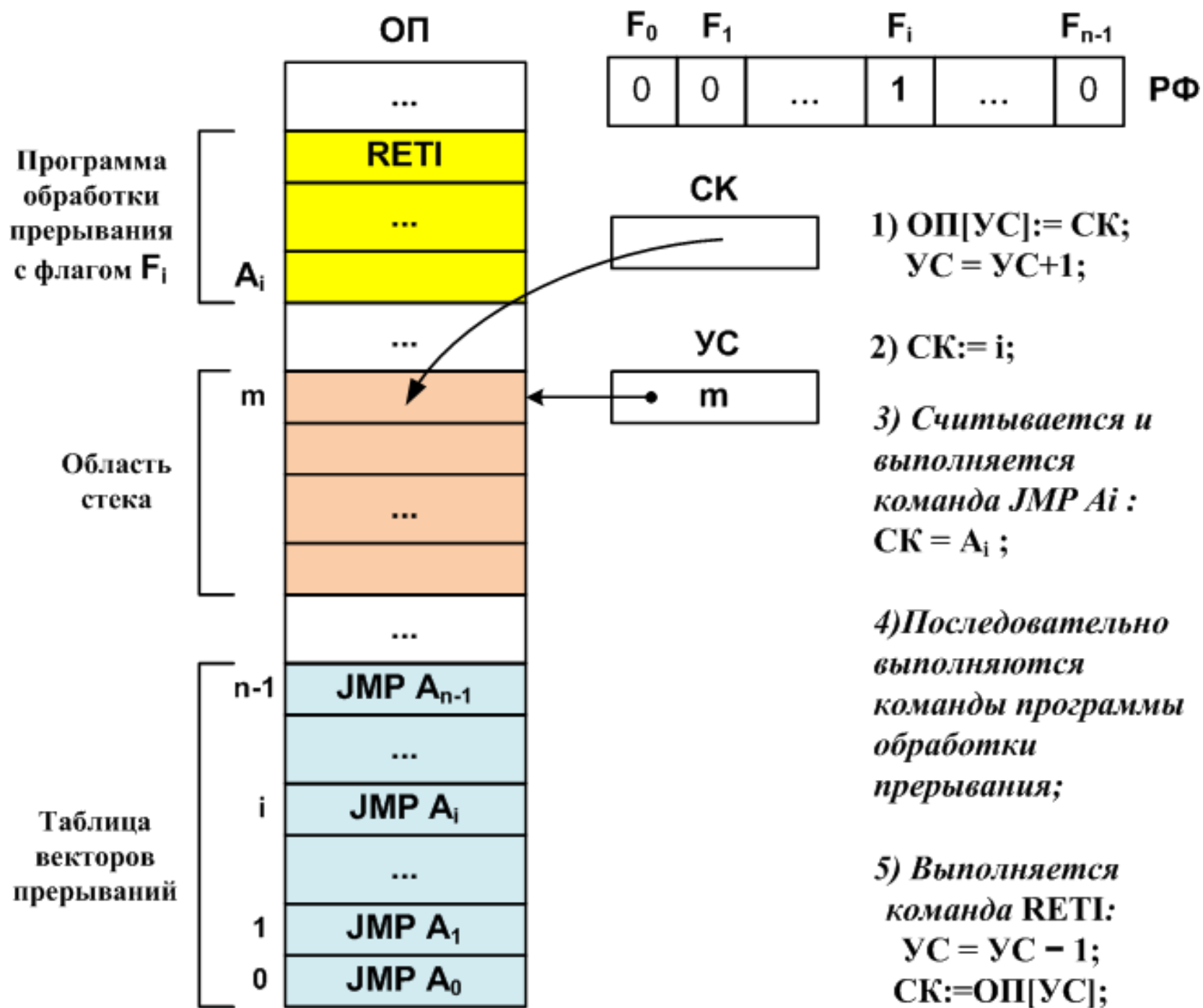
Промоделируйте
работу схемы,
например, для
переходов по

$>, \leq, \neq$.

Комбинационные схемы вычисления
осведомительных сигналов x_1, x_2, x_3
разработайте самостоятельно !

Переключения с программы на программу. Обработка прерываний

Для поддержки системы прерываний в состав процессора введены регистр флагов (РФ) и регистр-указатель стека (УС).



Во время выполнения программы может возникнуть прерывание, вызванное, например, исключительной ситуацией, произошедшей в процессе выполнения очередной команды (деление на ноль, переполнение с фиксированной точкой, исчезновение порядка, переполнение порядка, потеря значимости и.т.п.). Подобные прерывания называются исключениями и, как правило, не могут быть замаскированы (маскирование отменяет обработку прерывания).

Другой тип прерываний – прерывания, поступившие от внешних устройств (поступил запрос на ввод или вывод данных от внешнего устройства, пользователь нажал кнопку <Num Lock> на клавиатуре). Все наблюдали последнюю ситуацию: во время работы программы, например, текстового процессора WORD, при нажатии клавиши <Num Lock> загорается лампочка, т.е. прерывание от клавиатуры обрабатывается. Это значит, что во время работы программы Word осуществляется переход на программу обработки прерывания, которая и зажигает лампочку (или тушит, если клавиша <Num Lock> была нажата повторно). После выполнения программы обработки прерывания процессор возвращается к выполнению прерванной программы (Word).

Прерывания от внешних устройств могут быть замаскированы, т.е. их обработка может быть отменена.

Современные компьютеры работают в мультипрограммном режиме. Каждой из программ выделяется квант времени τ (например, $\tau=1/30$ с). По истечении этого кванта возникает прерывание от таймера, и процессор переключается на выполнение другой программы. Поскольку выделяемый квант мал, у пользователя создается иллюзия непрерывной работы его программы. Возврат осуществляется именно в то место программы, в котором она была прервана.

Аппаратная поддержка системы прерываний

Для поддержки системы прерываний процессор имеет два регистра специальных функций: регистр флагов (РФ) прерываний и регистр масок (РМ) прерываний. Число битов в этих регистрах соответствует числу источников (причин) прерываний, i -ый бит соответствует i -ой причине прерывания. Если в РМ процессора установить в единицу i -ый бит (это можно сделать специальной командой, которая есть в системе команд процессора), то соответствующее прерывание обрабатываться не будет. Как уже говорилось выше, не все прерывания могут быть замаскированы.

В ОП (если гарвардская архитектура, то в памяти программ) первые n слов отводятся под таблицу векторов прерываний (ТВП). Программист не должен их использовать при низкоуровневом программировании (на ассемблере или в машинных кодах). Трансляторы с языков высокого уровня их тоже не используют.

Каждому биту F_i в регистре флагов РФ, как уже говорилось, соответствует определенная причина прерывания и слово в ТВП.

В этом (i -ом) слове ТВП записана (в двоичном виде) команда безусловного перехода на адрес A_i первой команды программы обработки соответствующего прерывания, которая также хранится в ОП.

Для осуществления возможности возврата в определенное место прерванной программы (то, в котором она была прервана) в ОП выделяется определенная область под стек, а в структуру процессора вводится еще один специальный регистр – **указатель стека (УС)**.

В УС всегда находится адрес слова, которое является «верхушкой» стека (верхнего свободного элемента стека).

Стек – структура данных, реализующая принцип LIFO (last in, first out) – последний вошел, первый вышел. Если информация записывается в стек, то она записывается в «верхушку стека», т.е. в слово, адрес которого записан в УС. После этого адрес в УС изменяется (в нашей иллюстрации увеличивается) так, чтобы он вновь указывал на свободное слово (верхушку стека). При извлечении информации из стека сначала изменяется значение в УС (уменьшается в нашей иллюстрации) для того, чтобы из стека было извлечено значение, записанное последним.

После выполнения очередной команды процессор анализирует РФ. При обнаружении единицы в i -ом бите РФ ($F_i = 1$), производятся следующие действия:

1) В слово ОП, на которое указывает УС, копируется содержимое СК (счетчик команд указывает на следующую, еще не выполненную команду). После этого УС увеличивается на 1, чтобы указывать на свободное слово.

$$\text{ОП} [\text{УС}] = \text{СК} ;$$

$$\text{УС} = \text{УС} + 1 ;$$

2) В СК загружается адрес слова ТВП, соответствующего биту F_i (причине прерывания).

$$\text{СК} = i ;$$

3) Процессор выполняет команду, на которую указывает СК. В i -ом слове ОП находится команда безусловного перехода на адрес A_i первой команды программы обработки соответствующего прерывания.

$$\text{JMP } A_i ;$$

Результатом выполнения этой команды процессором является появление в СК адреса A_i : $\text{СК} = A_i$

4) Процессор последовательно выполняет команды программы обработки прерывания.

5) Последняя команда в программе обработки прерываний – команда возврата из прерывания RETI (Return Interrupt).

Она выполняется следующим образом: из стека извлекается запомненное значение адреса возврата и загружается в СК, работа процессора продолжается (довыполняется прерванная программа).

$$\begin{aligned} \text{УС} &= \text{УС} - 1; \\ \text{СК} &= \text{ОП} [\text{УС}]; \end{aligned}$$

Если после выполнения очередной команды в регистре флагов обнаружено несколько единиц, то выполняется та или иная процедура опроса прерываний с учетом или без учета их приоритетов , и все возникшие незамаскированные прерывания так или иначе обрабатываются (этот материал рассматривается в курсе «Операционные системы»).

Недостаток систем, основанных на векторах прерываний состоит в том, что запоминается только адрес возврата. Если для нормального довыполнения прерванной программы необходимо запомнить содержимое некоторых регистров специальных функций (РСФ) и регистров общего назначения (РОН) (они могут быть изменены программой обработки прерывания), то эта задача возлагается на программиста, который пишет программу обработки прерывания (при входе в программу обработки прерывания он сохраняет значения нужных регистров (например, регистра признака результата РПР) в ОП или в РОН, а перед командой RETI восстанавливает содержимое этих регистров, если оно было заporчено программой обработки прерываний.

Существуют системы , которые при возникновении прерывания сохраняют, а после обработки прерывания восстанавливают не только адрес возврата, но и значения некоторых регистров специальных функций, например, РПР, регистра масок прерываний и других специальных регистров, объединенных общим названием «Слово состояния программы» (ССП или PSW).